



SEMESTER PROJECT

Daniel Pereira Riquelme

A Software-Defined X-Band Implementation for Satellite Communications

Telecommunications Circuits Laboratory

Supervisor of the Project	Andreas Peter Burg
Co-Supervisor of the Project	Jonathan Magnin
Study Programme	Electrical Engineering

Lausanne, Suisse, 2026

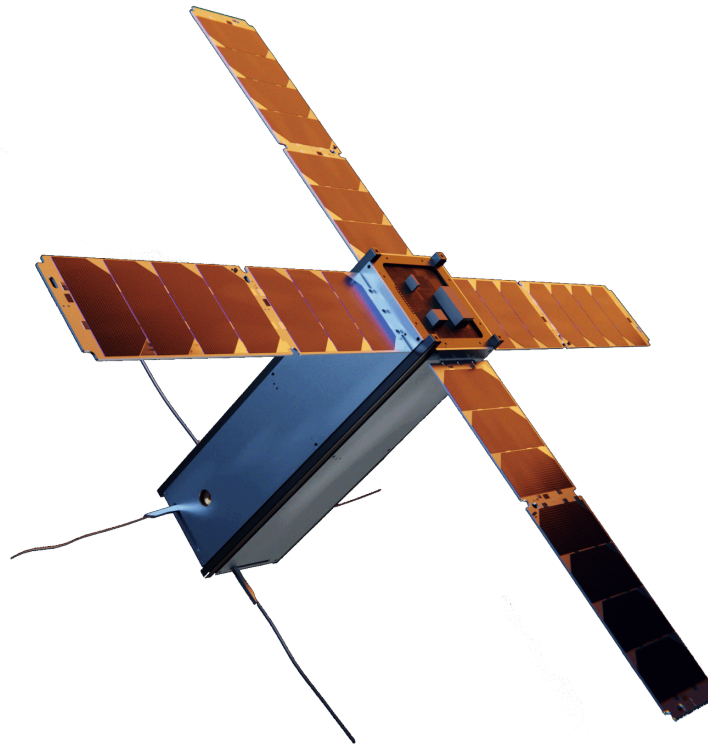
I would like to express my gratitude to Prof. Andreas Peter Burg for the opportunity to develop this project within the Telecommunications Circuits Laboratory. My deepest appreciation goes to my co-supervisor, Jonathan Magnin, for his outstanding day-to-day guidance, patience, and invaluable advice throughout our collaboration.

Throughout my studies at EPFL and UGR I had the pleasure to interact with many great lecturers. In particular, Chiurtu Nicolae, José Lizana and Ángel de la Torre left an unmistakable imprint on my work. I have also had the opportunity to belong to many associations with fascinating people, such as the Spacecraft Team within which this project was developed.

These years would not have been the same without my friends — Alex, Guille, Edu (and their amazing girlfriends), Jorge, Marco, Óscar (and also their amazing girlfriends), Marina, Miguel, Migue, Dimitri, Alex, Minas, among others — whose companionship, endless conversations, and all the times we couldn't stop laughing filled this time with joy and made it truly unforgettable. "Even some of them did the same in 24 hours as I did in a day".

I am especially grateful to my parents and family for their unwavering support, love, and, of course, their money. I don't think I'd be writing this without them, so all I can say is thank you.

Daniel Pereira Riquelme



Contents

Contents	iii
Introduction	v
Quick Summary	vi
List of Acronyms	vii
1 High-Rate Data Acquisition Pipeline	1
1.1 TM Synchronization and Channel Coding	1
1.2 Reed Solomon Coding	2
Mathematical Characterization 2 • Field and Code Generator Polynomials 2 • Symbol Interleaving 3	
1.3 Convolutional Coding	3
Distance Metrics and Performance Characteristics 4 • Punctured Architectures for Spectral Efficiency 4 • Optimal Detection 5	
1.4 Data Randomization (Pseudo-randomizer)	5
1.5 Physical Layer Architecture	6
Modulation and Spectral Containment 6 • Synchronization and Carrier Recovery 7	
2 GNU Radio Implementation	8
2.1 Design and Justification	8
3 Doppler Shift	11
3.1 A Model for the LEO Satellite Doppler Shift	11
3.2 Two-Stage Doppler Compensation	12
Coarse Compensation via Orbital Prediction 12 • Hardware Drifts and Residual Error Analysis 13 • Fine Frequency Tuning: The Costas Loop 14	
3.3 The Discrete Costas Loop	14
Phase Derotation 15 • Nonlinear Phase Detection and Error Metric Extraction 15 • The Loop Filter 16	
3.4 Costas Loop Characterization	16
Experiment 1: Transient Response 17 • Experiment 2: Frequency Acquisition Range 17 • Experiment 3: Frequency Tracking 18	
4 Frame Synchronization	21
4.1 Algorithms Already Implemented in GNU Radio	22
Phase Ambiguity 22	
4.2 Solving the Phase Problem.	23
4.3 Results	24
5 Link Budget and Final Results	25
5.1 Link Budget Analysis	25
5.2 Final Results and Real-Time Execution	26
6 A Case Study on QO-100 Narrowband Beacon	28
6.1 Ground Station Hardware Architecture.	28
6.2 The Es'hail-2 Satellite and Multimedia Beacon	29
6.3 Hardware Validation and Demodulation Results	30
A Physical Layer Error Probability	32
A.1 Optimal Detection for the Vector AWGN Channel	32

A.2 Error Probability for QPSK Modulation	33
B Block Diagrams	34
C Alternative Synchronization Architectures	37
C.1 Single-Branch Asynchronous Frame Synchronizer	37
Performance Limitations and Latency	37
References	39

I Introduction

Low Earth Orbit (LEO) satellite missions increasingly rely on high data rates to transmit large volumes of scientific information during their brief visibility windows. A typical LEO pass lasts only a few minutes, requiring ground stations to capture gigabits of data reliably using highly precise tracking antennas. However, the radio frequency (RF) signals received on the ground are significantly degraded by free-space path loss, thermal noise, and a severe, time-varying Doppler shift caused by the satellite's high relative velocity. To ensure data integrity, these communication links must employ robust synchronization techniques and advanced error correction codes.

Context and Motivation

The CHESS mission, developed by the [EPFL Spacecraft Team](#), exemplifies these challenges. Its upcoming CubeSat ([Pathfinder 0](#)) is currently under development and is equipped with a high-performance mass spectrometer that generates an unprecedentedly large volume of scientific data. To successfully retrieve this information, a high-rate X-band downlink is required.

While the mission's ground segment already possesses a 2.4-meter tracking antenna and actively develops its pointing software, it currently lacks the receiver capable of demodulating and decoding this high-rate telemetry. Traditionally, such receivers relied on rigid, hardware-based application-specific integrated circuits (ASICs). Today, Software-Defined Radio (SDR) technology allows us to replace dedicated silicon with digital signal processing algorithms running on standard commercial off-the-shelf (COTS) computer servers. The primary goal of this project is to design and implement this missing SDR receiver, bridging the gap between the space and ground segments while operating within the constraints of available computational resources and adhering to strict aerospace communication standards.

Project Objectives

The core objective of this project is to develop and validate a fully functional SDR receiver capable of reliably processing the satellite's high-rate X-band telemetry. To achieve this, the project is structured around the following specific goals:

- **Develop a Software-Defined Receiver:** Implement a modular SDR architecture capable of demodulating and decoding the telemetry in strict compliance with the CCSDS framework [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#).
- **Compensate for Massive Doppler Shift:** Design a tracking system to continuously estimate and eliminate the severe kinematic frequency shifts introduced by the LEO orbit [\[5\]](#) [\[6\]](#) [\[7\]](#).
- **Analyze Link Viability and Parameter Selection:** Perform link budget analyses and system simulations to evaluate theoretical performance metrics (such as Bit Error Rate curves) and optimize transmission factors [\[8\]](#) [\[9\]](#).
- **Achieve Real-Time High-Throughput Execution:** Optimize the SDR receiver software to execute seamlessly in real time on the base station server, sustaining a stable target throughput of 25 Mbps without frame loss.
- **Validate Hardware and Antenna Integration:** Verify the functional receiver and tracking setup by successfully capturing and decoding live telemetry beacons from the QO-100 narrowband satellite under real-world conditions [\[10\]](#).

Quick Summary

In **Chapter 1** we detail the CCSDS concatenated coding architecture (RS + Convolutional), pseudo-randomization, and baseband QPSK physical layer modeling.

In **Chapter 2** we present the end-to-end TX-RX simulation environment and validate our soft-decision decoding performance against standard curves.

In **Chapter 3** we evaluate a two-stage Doppler compensation strategy combining coarse open-loop orbital prediction with a fine closed-loop Costas loop.

In **Chapter 4** we resolve QPSK phase ambiguities and correlation floors by implementing a custom dual-branch feed-forward flywheel synchronizer.

In **Chapter 5** we analyze the CHESS mission link budget scenarios and benchmark the real-time 28 Mbps computational footprint on a base station server.

In **Chapter 6** we validate the physical ground station RF hardware chain by processing live DVB-S2 multimedia beacon signals from the geostationary Es'hail-2 satellite.



List of Acronyms

ADC	Analog-to-Digital Conversion
ASIC	Application-Specific Integrated Circuit
ASM	Attached Sync Marker / Access Sync Marker
AWGN	Additive White Gaussian Noise
BCH	Bose-Chaudhuri-Hocquenghem
BER	Bit Error Rate
BPSK	Binary Phase Shift Keying
CC	Convolutional Code
CCSDS	Consultative Committee for Space Data Systems
COTS	Commercial Off-The-Shelf
CRC	Cyclic Redundancy Check
DC	Direct Current
DSP	Digital Signal Processing
DVB-S2	Digital Video Broadcasting Satellite - Second Generation
ECEF	Earth-Centered Earth-Fixed
EIRP	Equivalent Isotropically Radiated Power
FEC	Forward Error Correction
FECS	Frame Error Control Field
FER	Frame Error Rate
FOM	Figure of Merit
GF	Galois Field
GS	Ground Station
HPBW	Half Power Beamwidth
IF	Intermediate Frequency
ISI	Inter-Symbol Interference
LDPC	Low-Density Parity-Check
LEO	Low Earth Orbit
LFSR	Linear Feedback Shift Register
LNB	Low Noise Block
MAP	Maximum A Posteriori
ML	Maximum Likelihood
NB	Narrowband
NCO	Numerically Controlled Oscillator
PD	Phase Detector

PFB	Polyphase Filter Bank
PI	Proportional-Integral
PLL	Phase-Locked Loop
QPSK	Quadrature Phase Shift Keying
RF	Radio Frequency
RS	Reed-Solomon
S/C	Spacecraft
SDR	Software-Defined Radio
SGP4	Simplified General Perturbations 4
SIMD	Single Instruction, Multiple Data
SNR	Signal-to-Noise Ratio
SRRC	Square Root Raised Cosine
TCA	Time of Closest Approach
TLE	Two-Line Element
TM	Telemetry
USB	Universal Serial Bus
XMLRPC	XML Remote Procedure Call

High-Rate Data Acquisition Pipeline

Before anything else, I would like to clarify that this project is exclusively dedicated to studying the Data Link layer and the Physical layer according to the OSI model.

There are different strategies for structuring reliable communication. The CHES Pathfinder-0 communication architecture strictly adheres to the standards **CCSDS publications** developed by the Consultative Committee for Space Data Systems (CCSDS). The physical layer is governed by [11], which defines X-Band modulation parameters, while [1] and [2] dictate the synchronization markers, FEC schemes such as Reed-Solomon and Viterbi, and the data link protocols essential for space-grade framing. The mentioned documents are the *Blue Books*. Theoretical rationale is derived from the *Green Books* [8], with operational best practices and administrative procedures guided by the *Magenta* and *Silver* series, respectively.

Although the ultimate goal of this project is to deploy a receiver, a complete transmitter pipeline was also implemented to generate valid test signals, simulate the transmission channel, and plot the necessary performance metrics.

1.1 TM Synchronization and Channel Coding

Following the CCSDS standard, in the synchronization and channel coding sublayer (*Blue Book* [11]), we must implement the blocks shown in Figure 1.1 → p.1 and Figure 1.2 → p.1. A concatenated coding configuration has been chosen, with Reed-Solomon as the outer code and a convolutional coder as the inner code. This is because it offers the highest code gain, and we will see in Chapter 5 → p.25 why it is the only viable combination.

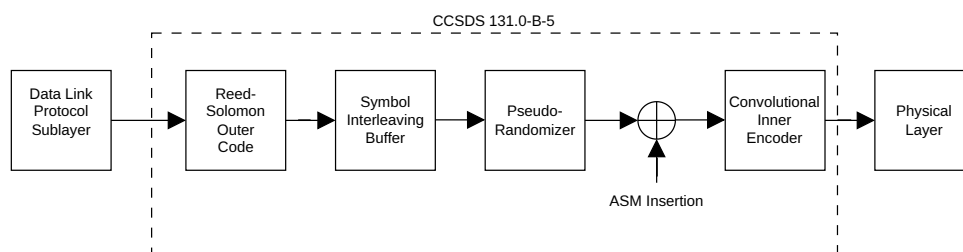


Figure 1.1 / Block diagram of the transmitter.

The concatenated configuration of Reed-Solomon (RS) and convolutional code (CC) provides a robust system with manageable decoding complexity. In this architecture, the inner convolutional code (typically processed by a Viterbi decoder) is responsible for correcting most of the channel errors, such as thermal noise (AWGN). However, by design, the Viterbi decoding process characteristically outputs residual errors in concentrated bursts.

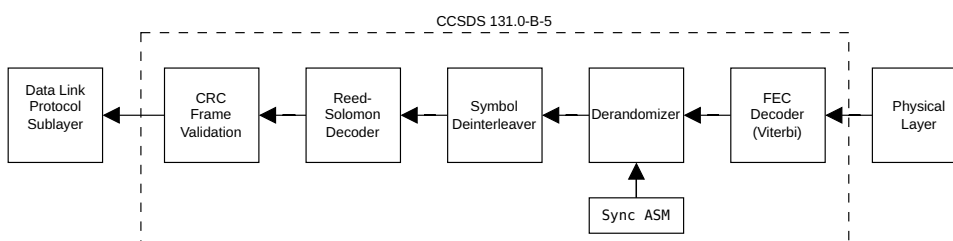


Figure 1.2 / Block diagram of the receiver.

This is where the outer Reed-Solomon code acts as the perfect complement to mitigate these bursts. Since the RS decoder processes information by symbols, the fact that residual Viterbi errors arrive clustered is highly advantageous: multiple erroneous bits concentrated in a single symbol count as just one symbol error for the RS code. Thus, the combination of both codes efficiently corrects both AWGN and burst errors without overwhelming the system's capacity.

1.2 Reed Solomon Coding

As introduced in [Section 1.1 ^{→ P.1}](#), the Reed-Solomon (RS) code acts as the outer error-correcting mechanism in the CCSDS telemetry architecture [\[1\]](#). While the inner Viterbi decoder efficiently processes additive white Gaussian noise (AWGN), as we mentioned, it characteristically outputs residual errors in concentrated bursts when metric crossovers occur (see [Figure 1.15 ^{→ P.6}](#)). RS codes are non-binary cyclic block codes optimally suited to mitigate this exact phenomenon, as their error-correction capability is independent of the number of bit errors contained within a single corrupted symbol.

Mathematical Characterization

The CCSDS standard specifies a systematic RS code defined over the Galois field $\mathbb{F}_{2^8}$ (or $\text{GF}(2^8)$). The fundamental building block is the J -bit symbol, where $J = 8$, resulting in an alphabet size of $2^8 = 256$ discrete elements [Section 5.1 \[8\]](#). The block length n of the codeword in symbols is dictated by the relationship $n = 2^J - 1$, yielding a standard unshortened length of $n = 255$ symbols.

To achieve maximum performance, the standard defines an information block size $k = 223$ symbols. The code's minimum distance d_{min} determines its error-correcting bounds:

$$d_{min} = n - k + 1 = 255 - 223 + 1 = 33 \quad (1.3)$$

From this distance metric, the maximum number of correctable symbol errors E per codeword is mathematically guaranteed as:

$$E = \left\lfloor \frac{d_{min} - 1}{2} \right\rfloor = 16 \quad (1.4)$$

Consequently, the systematic encoder appends $2E = 32$ parity check symbols to the original 223 information symbols, forming a total transmitted codeword of 2040 bits.

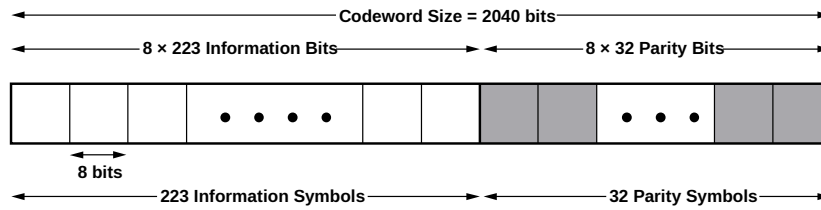


Figure 1.5 / Structure of the systematic CCSDS (255, 223) Reed-Solomon codeword, illustrating the payload and appended parity symbols.

Field and Code Generator Polynomials

The arithmetic operations governing the encoding process require a strictly defined finite field. The CCSDS standard constructs $\mathbb{F}_{2^8}$ using the following primitive field generator polynomial over $\mathbb{F}_2$:

$$F(x) = x^8 + x^7 + x^2 + x + 1 \quad (1.6)$$

Let α be a root of $F(x)$ such that $F(\alpha) = 0$. The specific code generator polynomial $g(x)$ over $\mathbb{F}_{2^8}$, which dictates the mapping of the systematic parity symbols, is constructed as a self-reciprocal

polynomial (palindrome) to minimize hardware multiplier complexity [Section 5.2 \[8\]](#):

$$g(x) = \prod_{i=128-E}^{127+E} (x - \alpha^{11j}) = \sum_{i=0}^{2E} G_i x^i \quad (1.7)$$

Furthermore, the standard mandates a **Dual Basis** symbol representation for transmission [Section 4.3.9 \[1\]](#). Instead of the conventional polynomial basis $(1, \alpha^1, \dots, \alpha^7)$, the symbols are transformed into a dual basis defined by the trace function $Tr(z)$. This specific algebraic mapping ensures standardized hardware compatibility across different mission architectures.

Symbol Interleaving

As previously established, the Viterbi decoder produces error bursts that can easily exceed the $E = 16$ symbol correction limit of a single RS codeword. To preserve the FEC integrity, the standard employs rectangular block interleaving with a depth $I \in \{1, 2, 3, 4, 5, 8\}$ [Section 4.3.5 [1]].

The interleaving process is mathematically equivalent to writing I consecutive RS codewords into an $I \times n$ matrix. The symbols are written row-by-row but transmitted column-by-column.

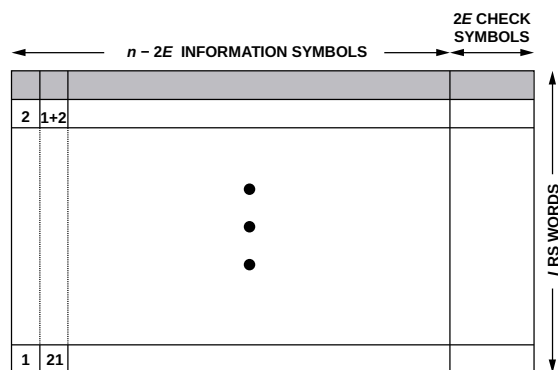


Figure 1.8 / Rectangular block interleaver matrix of depth I . A contiguous channel error burst of length B symbols is distributed upon de-interleaving such that no single codeword absorbs more than $\lceil B/I \rceil$ errors. You can find an example in [Figure B.2 $\rightarrow$ p.36].

Upon reception, the de-interleaver performs the inverse transposition. By spreading a contiguous burst of $\mathcal{B}$ channel errors across I distinct codewords, the effective number of errors per codeword is reduced to approximately $\mathcal{B}/I$. As long as the degraded symbols per row remain $\leq E$, the Reed-Solomon algebraic decoder can successfully reconstruct the original transmitted frame without data loss.

1.3 Convolutional Coding

The convolutional coding scheme established for space telemetry [Section 3 [1]] is defined as a linear mapping over the Galois field $\mathbb{F}_2$. This specific implementation is a non-systematic, transparent code with a constraint length $K = 7$ and a fundamental code rate $r = 1/2$, [Figure 3.1 $\rightarrow$ p.11]. The mathematical structure of the encoder is best described in the transform domain using the delay operator D , where the input sequence $\mathbf{u}(D)$ is transformed into an output codeword $\mathbf{v}(D)$ through the generator matrix $\mathbf{G}(D) = [G_1(D), G_2(D)]$.

The generator polynomials, expressed in their octal representation as 171_8 and 133_8 , are defined by the following structural equations:

$$\begin{aligned} G_1(D) &= 1 + D + D^2 + D^3 + D^6 \\ G_2(D) &= 1 + D^2 + D^3 + D^5 + D^6 \end{aligned}$$

The encoding process is mathematically defined as the discrete convolution of the input sequence $\mathbf{u} \in \mathbb{F}_2^\infty$ with the impulse responses $\mathbf{g}_i$ of the generator polynomials, where for each information bit u_k at time k , the encoder generates a raw symbol pair $(c_{1,k}, c_{2,k})$ according to the linear mapping

$$c_{i,k} = \sum_{j=0}^{K-1} u_{k-j} \cdot g_{i,j} \pmod{2}. \quad (1.9)$$

While these generator outputs $\mathbf{c}_i(D) = \mathbf{u}(D)G_i(D)$ establish the error-correcting structure, the standard mandates a non-linear **Inversion** transformation specifically on the second stream to guarantee a high Transition Density within the channel. Consequently, the final transmitted symbols $v_{k,1}$ and $v_{k,2}$ are formulated as $v_{k,1} = c_{1,k}$ and $v_{k,2} = c_{2,k} \oplus 1$, a structural modification that ensures the sequence $\mathbf{v}$ remains alternating even during periods of null input data ($u_k = 0$), thereby providing the receiver's symbol synchronizer with the necessary edges to maintain a stable lock in the extremely low SNR conditions characteristic of deep space missions.

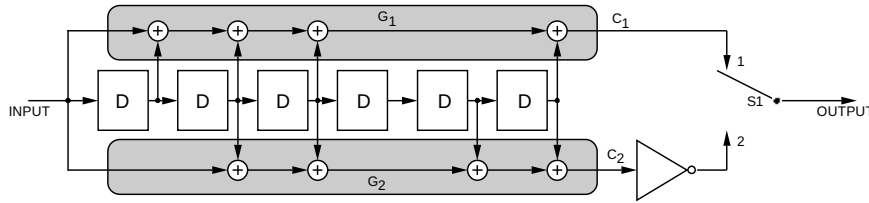


Figure 1.10 / The CCSDS Standard Convolutional Code: Constraint Length $K=7$, Rate $1/2$.

The code $\mathcal{C}$ is inherently transparent, a property defined as $f(\overline{\mathbf{u}}) = \overline{f(\mathbf{u})}$. This characteristic allows the *Viterbi* decoder to resolve the 180° phase ambiguity inherent in BPSK and QPSK modulations. If the demodulator captures the signal with inverted polarity, the decoder produces the inverted version of the original data, which can then be corrected by the frame layer without losing the coding gain.

Distance Metrics and Performance Characteristics

The error-correcting capability of the code is determined by its minimum free distance, d_{free} , which represents the minimum Hamming distance between any two distinct infinite-length paths in the *trellis* diagram. For the $K = 7$ code, the d_{free} is exactly 10^1 . From this metric, we derive the maximum number of symbol errors t that the decoder is guaranteed to correct within a specific observation window:

$$t = \left\lfloor \frac{d_{free} - 1}{2} \right\rfloor = \left\lfloor \frac{10 - 1}{2} \right\rfloor = 4 \quad (1.11)$$

In the presence of additive white Gaussian noise (AWGN), the asymptotic coding gain is proportional to rd_{free} . For this $r = 1/2$ code, the distance properties provide a significant reduction in the required E_b/N_0 to achieve a target bit error rate, typically yielding a gain of approximately 5.5 dB at a BER of 10^{-5} when compared to uncoded BPSK.

Punctured Architectures for Spectral Efficiency

When bandwidth constraints require a higher spectral efficiency η than what the $r = 1/2$ code provides, the standard employs a puncturing operator P_μ [Section 4.3 [8]]. This process involves the systematic deletion of symbols from the output of the rate $1/2$ encoder according to a puncturing pattern $\mathbf{m}$, enabling alternative code rates such as $r \in \{2/3, 3/4, 5/6, 7/8\}$. However, a rate of $1/2$ is chosen to improve signal integrity.

¹Exhaustive searches for rate $1/2$ codes confirm that $d_{free} = 10$ is the absolute mathematical maximum achievable for a constraint length $K = 7$. This specific code was selected as a local optimum because it maximizes error-correcting performance while remaining within the $2^{K-1} = 64$ state computational limit of the Viterbi decoders available during its standardization. You can find more information here [Section 4.2 [8]].

Optimal Detection

The optimal detection of convolutional codes is mathematically formulated as a Maximum Likelihood sequence estimation problem where the objective is to minimize the probability of word error by selecting the most probable path through the *trellis*, see [Figure 1.14 → p.5](#). Given a received sequence $\mathbf{Z}$, the decoder identifies the message $\mathbf{U}^{(m')}$ that maximizes the conditional probability density function, or likelihood function, which is formally expressed as:

$$P(\mathbf{Z}|\mathbf{U}^{(m')}) = \max_{\text{all } \mathbf{U}^{(m)}} P(\mathbf{Z}|\mathbf{U}^{(m)}) \quad (1.12)$$

In the presence of additive white Gaussian noise (AWGN), maximizing this likelihood is equivalent to minimizing the cumulative squared Euclidean distance between the received symbols z_{ji} and the candidate codeword symbols $v_{ji}^{(m)}$. The total path metric $\mathcal{M}$ for a sequence of length $\mathcal{L}$ symbols is computed as follows:

$$\mathcal{M} = \sum_{j=1}^{\mathcal{L}} \sum_{i=1}^n (z_{ji} - v_{ji}^{(m)})^2 \quad (1.13)$$

This global optimization is efficiently executed by the **Viterbi** algorithm, which decomposes the problem into a recursive series of local decisions based on the state transitions of the encoder.

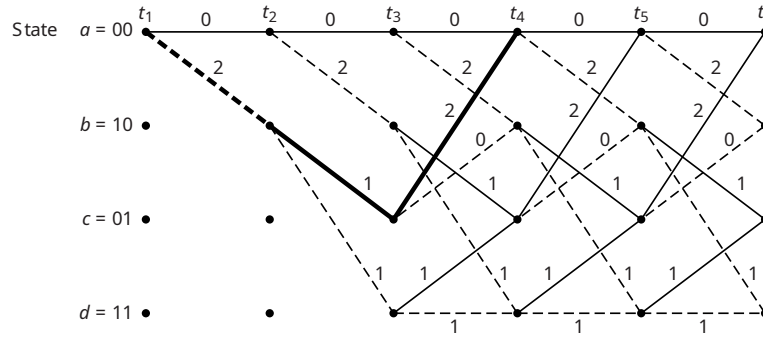


Figure 1.14 / Example of Trellis diagram [Figure 7.7 \[12\]](#), labeled with distances from the all-zeros path. $K=3$.

A significant characteristic of the **Viterbi** decoding process is the nature of its residual errors, which typically appear as error bursts rather than independent bit errors. These bursts occur when the channel noise is sufficient to cause a metric crossover, forcing the decoder to deviate from the correct path and follow an incorrect branch until the cumulative metrics allow the paths to re-merge. Because these error bursts are statistically correlated and span several constraint lengths, they can significantly degrade the performance of subsequent protocols that assume independent error distributions. This necessitates the use of a **Frame Error Control Field (FECF)** based on a cyclic redundancy check (CRC) or the concatenation with an outer block code like Reed-Solomon to ensure the integrity of the decoded data frames.

In [Figure 1.15 → p.6](#), you can see an example of what happens at low SNR. The plot shows different frames (in rows) found by the synchronization algorithm after the **Viterbi** decoder. You can see how errors (in red) mostly occur in bursts; that's when the decoder takes the wrong path and takes a few bits to get back on track. When these error bursts hit the ASM frame, they destroy it, making it hard to recover the frame and leading to those gray gaps in the image.

1.4 Data Randomization (Pseudo-randomizer)

Despite the excellent error-correcting properties of the analyzed codes, these linear schemes cannot guarantee on their own a sufficient number of bit transitions in the communication

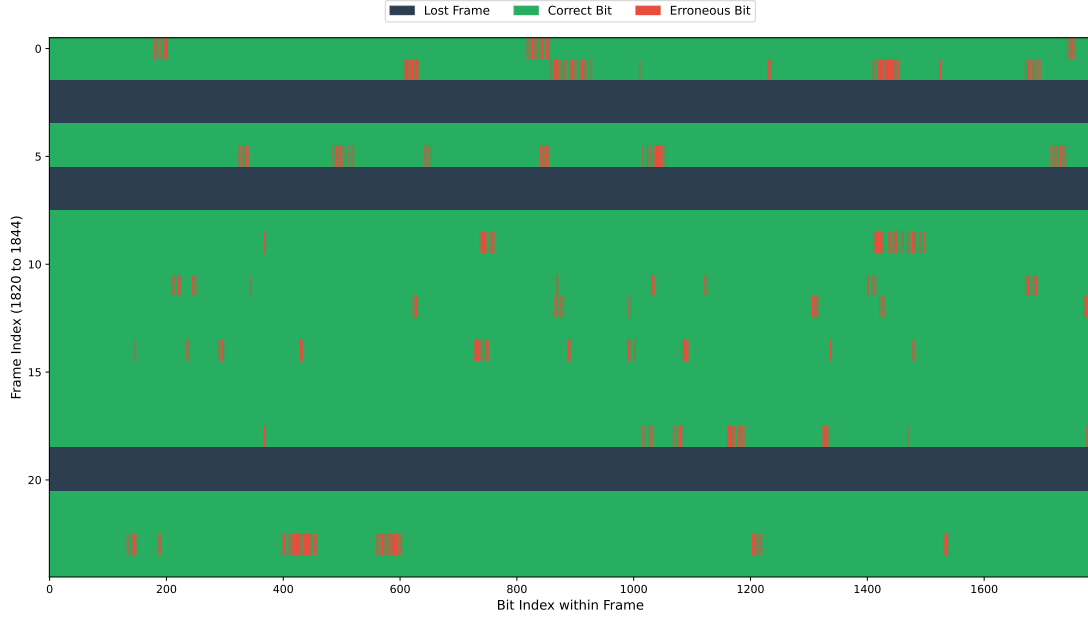


Figure 1.15 / 2D Map: Bit Errors and ASM Loss when $E_b/N_0 = 3.8$ dB.

channel. The presence of prolonged sequences of constant bits (such as contiguous bursts of zeros or ones) critically degrades the performance of clock recovery devices at the receiver, causing symbol synchronizers to lose phase lock. To ensure a high bit transition density, mitigate the formation of discrete spectral lines in the radiated signal, and facilitate rapid carrier acquisition, the CCSDS standard mandates the application of a pseudo-randomization block (Pseudo-randomizer) to the data sequence prior to transmission.

The implementation of this subsystem is conventionally structured using an 8-stage linear feedback shift register LFSR, governed by the irreducible generator polynomial $h(x) = x^8 + x^7 + x^5 + x^3 + 1$ over the Galois field $\mathbb{F}_2$. This circuit generates a deterministic pseudo-random sequence with a maximum period of 255 bits that is combined with the structured codeblock via a bit-by-bit XOR operation. Due to the symmetric nature of this algebraic transformation, the process is completely reversible at the receiving end.

1.5 Physical Layer Architecture

The physical layer is responsible for the continuous-time representation of the discrete symbol stream, translating logical states into propagating electromagnetic waveforms. In adherence to the stringent spectral occupancy regulations defined by the CCSDS for X-Band communications [11], the communication architecture relies on a Quadrature Phase Shift Keying (QPSK) modulation scheme, strictly governed by band-limiting pulse shaping techniques. This section delineates the mathematical models and digital signal processing blocks implemented within the continuous-time baseband architecture.

Modulation and Spectral Containment

To mitigate adjacent channel interference and satisfy the maximum bandwidth constraints without introducing Inter-Symbol Interference (ISI), the system employs a Square Root Raised Cosine (SRRC) filter [13]. The CCSDS Blue Book Section 2.4.18 [11] enforces the use of SRRC filtering to achieve optimal spectral containment for high-rate data transmissions. Let T_s denote the channel symbol duration. The frequency response of the SRRC filter, $H(f)$, is

mathematically parameterized by the roll-off factor α , and is formulated as:

$$H(f) = \begin{cases} 1, & |f| < \frac{1-\alpha}{2T_s} \\ \sqrt{\frac{1}{2} \left[1 + \sin\left(\frac{\pi T_s}{\alpha} \left(\frac{1}{2T_s} - |f| \right)\right) \right]}, & \frac{1-\alpha}{2T_s} \leq |f| \leq \frac{1+\alpha}{2T_s} \\ 0, & |f| > \frac{1+\alpha}{2T_s} \end{cases} \quad (1.16)$$

In this specific implementation, the roll-off factor is established at $\alpha = 0.5$ (`excess_bw = 0.5`), meticulously balancing the trade-off between spectral efficiency and timing jitter immunity. The digital QPSK modulator maps the coded data into complex baseband symbols $s_k \in \{\pm 1 \pm j\}$, which are subsequently up-sampled by a factor of $SPS = 2$ and convolved with the discrete-time SRRC impulse response.

Synchronization and Carrier Recovery

Following the optimal minimum-distance detector rationale [14], the receiver must flawlessly estimate both the symbol timing and the residual carrier phase to accurately project the received vector onto the QPSK decision regions. To resolve the fractional delay embedded in the asynchronous sampling process, a Polyphase Filter Bank (PFB) clock synchronizer is utilized. This algorithm employs a Maximum Likelihood derivative matched filter to track the timing error, continuously adjusting the decimated sample selection through a closed-loop interpolator [13].

Once the optimal symbol epochs are recovered, the residual phase offsets and subtle frequency drifts are tracked utilizing a Costas loop configured for fourth-order symmetry. This is detailed in [Chapter 3 ^{→ p.11}](#).

GNU Radio Implementation

Traditionally, X-band satellite reception relied on expensive, fixed-function hardware. Today, Software-Defined Radio (SDR) provides a more flexible approach by shifting the core signal processing from physical integrated circuits to software algorithms. This chapter details the construction of the SDR receiver architecture—referred to throughout this report as the signal processing *pipeline*¹.

A critical constraint defining this architecture is real-time execution. Every processing stage, from Doppler estimation to Viterbi decoding, must be computationally efficient and perfectly synchronized. The main challenge lies not only in demodulating the signal, but in optimizing the code to process the high data rate without buffer overflows or frame loss, effectively keeping pace with the continuous RF capture.

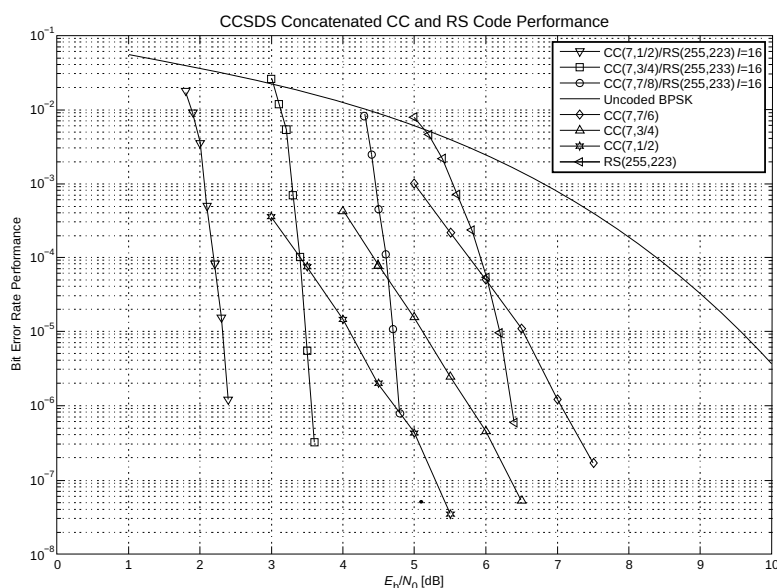


Figure 2.1 / Performance of Concatenated Coding Systems with Ideal Interleaving, $E=16$, Punctured Codes [Figure 6-4 in [8]].

2.1 Design and Justification

The GNU Radio community is highly active, resulting in a substantial body of open-source code that provides software-defined solutions for applications akin to the one addressed in this report. Regarding the CCSDS suite of protocols, several libraries have been instrumental to our work, such as `gr-satellites` [3], `gr-highdatarate-modem` [4], `gr-ccsds` [15], and the native GNU Radio blocks themselves (Block Docs). As previously established in Chapter 1 ^{→ P.1}, fulfilling the core objectives of this project requires evaluating the receiver against a complete TX-RX simulation chain. By leveraging these libraries, we constructed this full simulation environment to generate our BER curves and rigorously characterize the system's performance.

To achieve this, we utilized all the aforementioned libraries (except for [15], as its blocks are outdated). The most widely adopted library is [3], but it primarily provides blocks for RX. Furthermore, during our evaluation, we found that some blocks did not perform as expected,

¹Throughout this report, the terms *pipeline* and *flowgraph* are used interchangeably to denote the GNU Radio architecture—a sequence of interconnected Digital Signal Processing (DSP) modules designed to transform a raw stream of digitized Radio Frequency (RF) samples into decoded telemetry.

others were only partially implemented, and a few did not strictly adhere to the standard. We will discuss a major missing implementation issue within these libraries later in [Chapter 4](#) [→ p.21](#). Consequently, by integrating blocks from various libraries and programming new custom blocks ourselves, we successfully developed a flowgraph that operates in compliance with CCSDS specifications. In the appendix, you can refer to [Figure B.1](#) [→ p.35](#), which illustrates the blocks we programmed alongside those reused from existing libraries. These custom blocks can be found in the following library [SDR-Xband-receiver](#).

It is worth noting that the simulations found in the CCSDS documentation represent ideal performance curves. According to the information provided in the *Green Books*, the performance depicted in [Figure 2.1](#) [→ p.8](#) assumes an AWGN channel, perfect symbol and frame synchronization, and an ideal pulse-shaping filter. These assumptions are made to demonstrate the maximum theoretical capability of these mathematical codes.

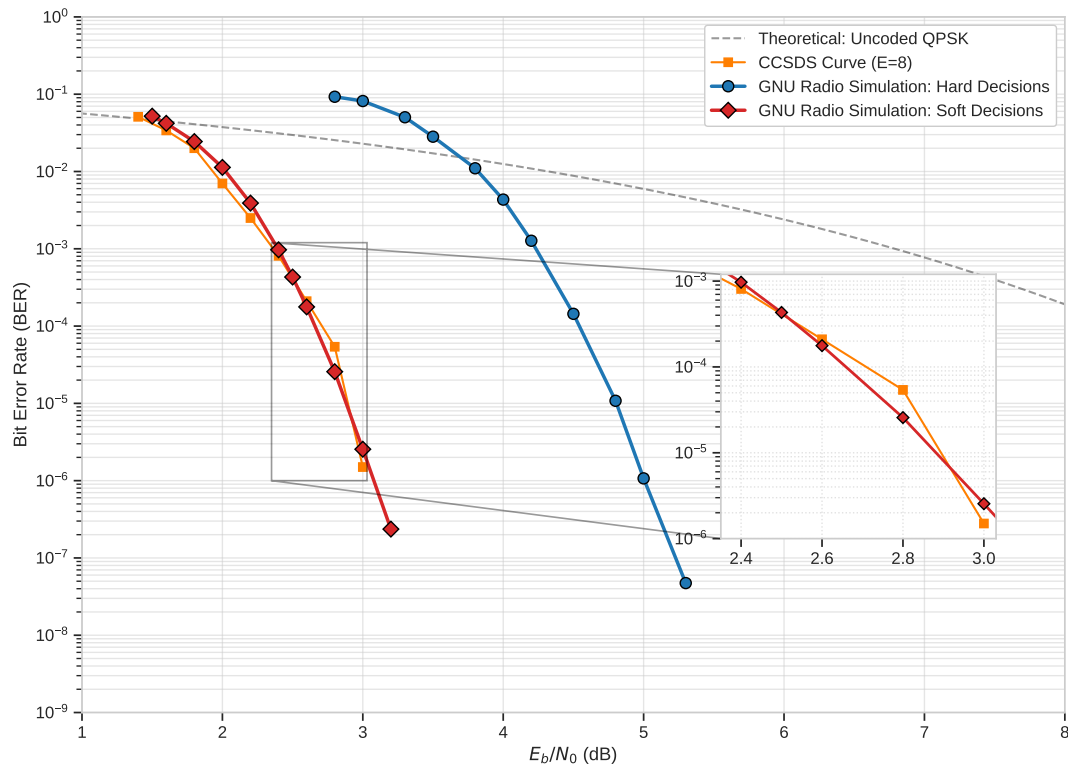


Figure 2.2 / Simulated curve in GNU Radio vs. CCSDS curve found in [Figure 6-9 \[8\]](#) for an Interleaving Depth of 1 and $E=8$.

Taking this into account, we simulated the system to obtain [Figure 2.2](#) [→ p.9](#) and observed that our result is practically identical to the standard's theoretical curve. In fact, our curve appears smoother, likely due to the higher number of simulated frames. Additionally, we plotted the system's performance assuming hard decisions to serve as a baseline. As established in classical communication theory, hard-decision decoding typically introduces a performance degradation of approximately 2 dB compared to unquantized soft decisions. However, this plot is strictly illustrative; our final receiver architecture exclusively employs soft-decision decoding to maximize the coding gain.

However, these ideal graphs are insufficient to evaluate the practical implementation of the receiver. In the subsequent chapters, we will introduce a complete channel simulation model to examine the operational challenges of the link. Specifically, we will analyze the impact of the Doppler effect and demonstrate how to develop a robust algorithm to solve the frame synchronization problem. By integrating these physical impairments and their respective tracking

mechanisms into the simulation, we will generate more realistic BER curves. As we incorporate Doppler compensation and real-time frame synchronization, the BER will naturally increase when compared to the purely ideal curves simulated previously.

Doppler Shift

Satellite RF signals are shifted in frequency proportionally to the line-of-sight velocity between the satellite and ground station, due to the Doppler effect. The Doppler frequency depends on time, on the location of the ground station, and on the orbit of the satellite, as well as on the carrier frequency [6]. In satellite communications, it is common to correct for the Doppler present in the downlink signals before processing them. It is also common to correct for the uplink Doppler before transmitting an uplink signal, so that the satellite receiver sees a constant frequency.

3.1 A Model for the LEO Satellite Doppler Shift

In Low Earth Orbit (LEO) satellite communications, the kinematic dynamics between the spaceborne platform and the terrestrial station introduce a pronounced, time-varying frequency translation governed by the Doppler phenomenon. To characterize this effect without the computational burden of iterative numerical trajectory propagation, an elegant analytical framework can be derived by applying spherical trigonometry under valid orbital simplifications (see this article [5]). Assuming a circular orbit, the trajectory of the satellite during its short visibility window can be tightly approximated by a great-circle arc in the Earth-Centered Earth-Fixed (ECEF) reference frame. Furthermore, the tangential velocity magnitude in the ECF frame exhibits minor variations and is effectively approximated by the constant angular velocity at the peak latitude, defined as $\omega_F \approx \omega_s - \omega_E \cos i$, where ω_s denotes the inertial orbital angular velocity, ω_E represents the Earth's rotation rate, and i is the orbital inclination.

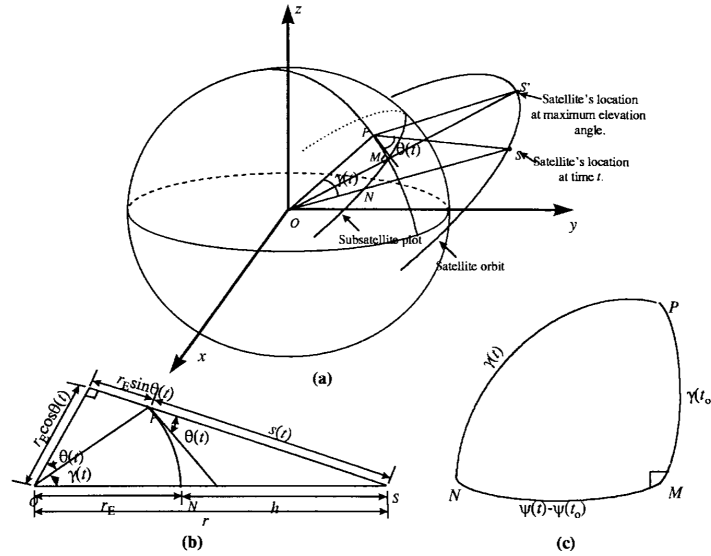


Figure 3.1 / Satellite geometry during visibility window at location P on earth. (a) Basic satellite geometry. (b) Plane triangle SOP. (c) Spherical triangle MNP. [Fig. 2 [5]]

By applying the law of cosines on the plane triangle formed by the Earth's center, the ground terminal, and the satellite's position, alongside the spherical cosine law for the projection of the ground trace, the dynamic slant range $s(t)$ and its time-derivative $\dot{s}(t)$ can be resolved analytically as a function of time t . Consequently, the normalized Doppler shift, formulated as $\frac{\Delta f(t)}{f_0} = -\frac{1}{c} \dot{s}(t)$, yields the following closed-form expression (Equation 5 in [5]):

$$\frac{\Delta f(t)}{f_0} = -\frac{1}{c} \frac{r_E r \sin(\omega_F t) \cos \gamma_0 \cdot \omega_F}{\sqrt{r_E^2 + r^2 - 2r_E r \cos(\omega_F t) \cos \gamma_0}} \quad (3.2)$$

Within this analytical formulation, f_0 denotes the nominal downlink carrier frequency — centered at 10.475 GHz for the X-band system under analysis — while c , r_E , and r represent the speed of light, the mean radius of the Earth, and the orbital radius ($r = r_E + h$, where h is the satellite altitude), respectively. The temporal variable t is defined relative to the zero-Doppler instant ($t = 0$), which geometrically corresponds to the epoch of closest approach and maximum elevation. The constant angular distance γ_0 represents the central angle at the maximum elevation point, implicitly determined via the transcendental relation $\cos(\theta_{\max} + \gamma_0) = \frac{r_E}{r} \cos \theta_{\max}$, where $\theta_{\max}$ is the maximum elevation angle achieved during the specific pass. Equation (3.2) $\rightarrow$ p.11 mathematically outlines a symmetric S-curve whose slope at the inflection point is explicitly parameterized by $\theta_{\max}$.

3.2 Two-Stage Doppler Compensation

As established by the analytical model, the macroscopic kinematic Doppler shift during an X-band pass can span hundreds of kilohertz. However, relying solely on an open-loop analytical model or orbital predictions is insufficient for maintaining a robust communication link. The combination of hardware oscillator drifts, Low Noise Block (LNB) offsets, and kinematic mismatches due to aging Two-Line Elements (TLEs) dictates that applying a perfectly inverted theoretical S-curve will never perfectly center the signal at baseband. A residual, time-varying Doppler shift will always persist.

To overcome this unpredictability and ensure continuous synchronization, our Software-Defined Radio (SDR) architecture implements a two-stage Doppler compensation strategy: a *coarse* open-loop estimation followed by a *fine* closed-loop tracking mechanism.

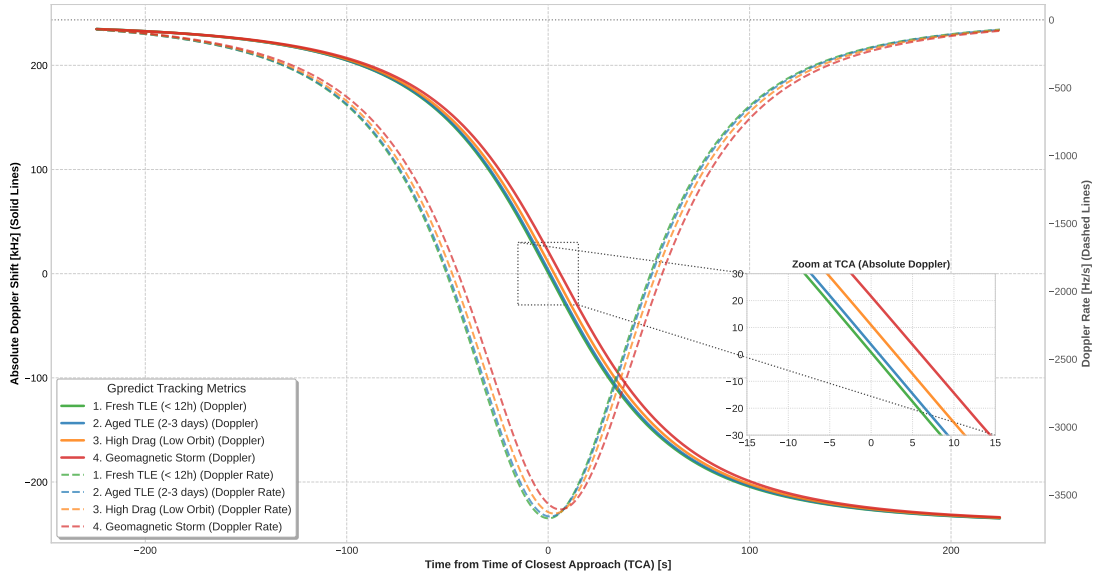


Figure 3.3 / Doppler Curves & Rates vs TLE Conditions.

Coarse Compensation via Orbital Prediction

The first stage involves removing the bulk of the predictable frequency shift. To achieve this, we rely on an open-loop coarse correction using Two-Line Elements (TLEs) to predict the satellite's altitude and position. A TLE set encodes the orbital elements of an Earth-orbiting object at a specific epoch. However, due to the limited number of decimals in the format and the underlying mathematical limitations of the SGP4 propagation model, even a freshly issued TLE possesses an inherent positional average error of approximately 3 kilometers [16]. Because these ephemerides inevitably correspond to an orbit in the past compared to the true physical

orbit at the time of reception, kinematic mismatches always arise.

To process these orbital parameters dynamically, we utilize Gpredict, a highly regarded open-source real-time satellite tracking and orbit prediction application. Gpredict [7] continuously calculates the expected Doppler shift using the SGP4 propagation model based on the freshest available TLEs and the ground station's geographical coordinates. While implementing an SGP4 orbital propagator natively within the GNU Radio flowgraph might initially seem more integrated, outsourcing this task to an external tool offers crucial architectural advantages. Specifically, it decouples the low-rate orbital mechanics calculations from the high-speed DSP pipeline. This separation ensures that the computational resources within the flowgraph are strictly dedicated to sustaining the high 25 Mbps real-time demodulation rate without risking buffer overflows.

Gpredict interfaces with the GNU Radio flowgraph via a network socket connection, targeting an XMLRPC Server block. As the satellite passes overhead, Gpredict acts as the master controller, periodically sending target frequency update commands to a local oscillator (Signal Source) or a frequency-shifting block inside GNU Radio. This dynamically shifts the incoming RF stream in the opposite direction of the predicted Doppler curve.

Hardware Drifts and Residual Error Analysis

While the coarse correction effectively removes the macroscopic Doppler shift, real-world links suffer from unpredictable hardware deviations that add up to the TLE kinematic mismatch. Both the satellite's transmitter and the receiver's Low Noise Block (LNB) introduce thermal and long-term frequency drifts. For example, LNB offsets can easily introduce several kilohertz of baseline error. To simulate these hardware effects accurately, the system parameters have been modeled taking as reference the official datasheets of said components [10].

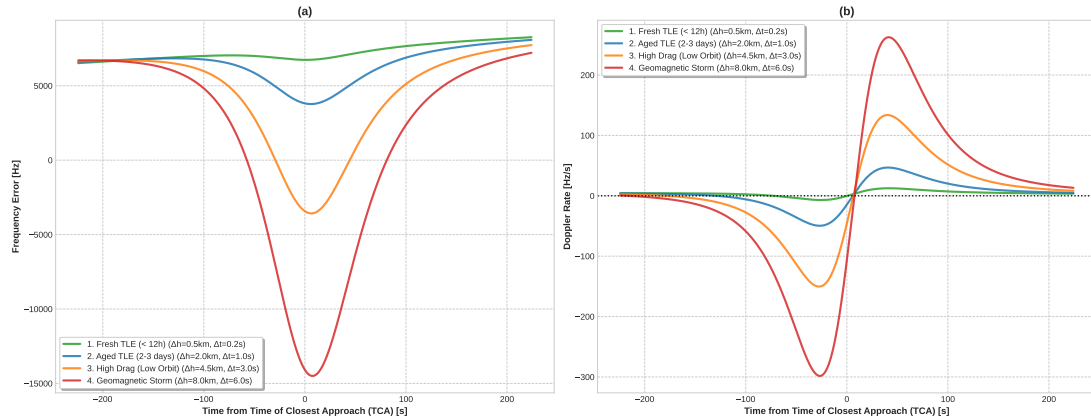


Figure 3.4 / (a) Total Residual Error vs TLE Accuracy. (b) Derivative of Residual Error (Doppler Rate).

To fully characterize the system's robustness, several operational cases have been considered that represent different degrees of degradation in the orbital information, as illustrated in Figure 3.3 ^{→ p. 12}. These simulated scenarios range from a nominal situation with recently updated TLEs (less than 12 hours old) to the standard degradation of the data after a couple of days. Furthermore, we model extreme situations such as high atmospheric friction or geomagnetic storms that induce delays of several seconds in the satellite's trajectory¹.

These simulations allow us to deeply evaluate the impact of altitude and time deviations on

¹For further context on these orbital perturbations, one can look at space weather reports [Space Weather Conditions NOAA](#), which discuss how events like geomagnetic storms increase atmospheric drag. A historical example is NASA's Solar Maximum Mission (SMM) spacecraft, which was reported to have "dropped as if it hit a brick wall" due to sudden atmospheric expansion.

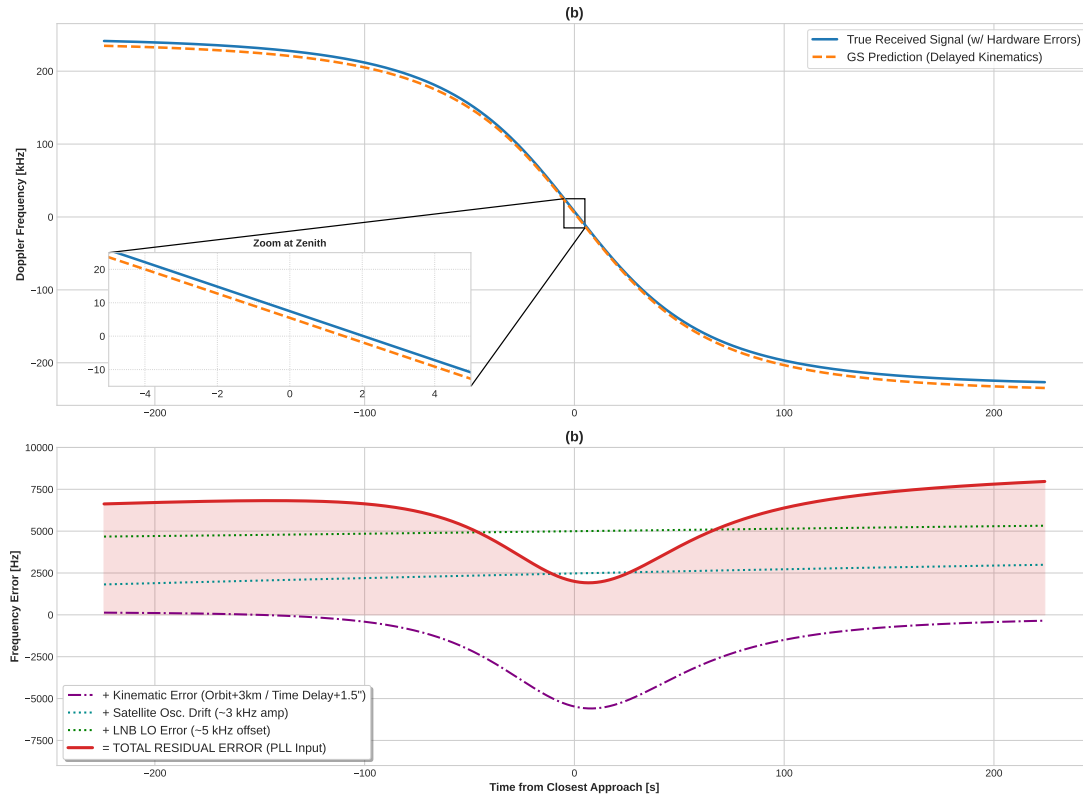


Figure 3.5 / A particular case of a Coarse Doppler Correction. (a) True vs Predicted Doppler. (b) Residual Error after coarse correction (What the Modem Actually Sees).

the total frequency residual and the Doppler rate of change, which are critical parameters to guarantee receiver lock during the pass, as seen in [Figure 3.4 → p.13](#).

To visualize the practical outcome of this first stage, [Figure 3.5 → p.14](#) illustrates a particular scenario of coarse Doppler correction. Part (a) plots the true received signal against the delayed kinematic prediction provided by Gpredict. Because the coarse stage relies on this imperfect prediction, it inherently leaves behind a dynamic residual frequency error. This leftover error, depicted in [Figure 3.5 → p.14](#)(b), represents exactly what the software modem sees after the coarse compensation has been applied.

Fine Frequency Tuning: The Costas Loop

Because the coarse stage leaves behind this dynamic residual frequency error — which fluctuates unpredictably over time due to the combination of TLE aging, SGP4 limitations, and hardware oscillator drifts — a secondary tracking stage is strictly required to reliably demodulate the downlink. To finely tune the frequency, eliminate the residual offset shown in [Figure 3.5 → p.14](#)(b), and guarantee phase lock during the pass, we introduce a second step: a discrete Costas loop designed to autonomously track and compensate for these residual fluctuations. This closed-loop estimation block will be addressed in detail in [Section 3.3 → p.14](#).

3.3 The Discrete Costas Loop

In order to perform a fine frequency estimation, an element that tracks the frequency in real-time to correct it is necessary. Furthermore, this algorithm must be efficient since it will have to work with several million samples per second. Following a coherent demodulation architecture, the Costas Loop implemented in GNU Radio [Costas Loop Block](#) represents effective solution to

this problem. This block is well-established within the community and fully optimized for this function. The objective of this exposition is to provide a rigorous mathematical characterization of the entire block diagram, correlating each theoretical discrete-time equation directly with its corresponding C++ implementation. The mathematical representation of a phase-tracking loop and its subsequent simulations, as referenced in [Figure 3.6](#), will allow us to precisely deduce its behavior and performance as well as its stability.

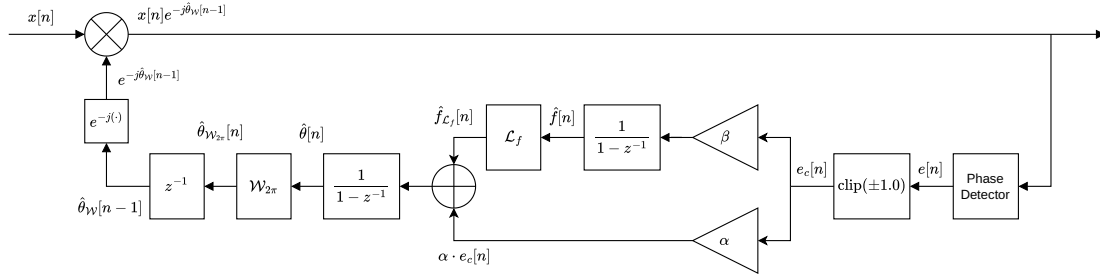


Figure 3.6 / Diagram of the **Costas Loop Block** coded by GNU Radio.

For each incoming discrete sample, the internal state vector $\mathbf{v}[k] = [\hat{f}[k], \hat{\theta}[k]]^T$ is updated according to the following state-space equation:

$$\begin{bmatrix} \hat{f}[k+1] \\ \hat{\theta}[k+1] \end{bmatrix} = \begin{bmatrix} \mathcal{L}_f & 0 \\ 0 & \mathcal{W}_{2\pi} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} \hat{f}[k] \\ \hat{\theta}[k] \end{bmatrix} + \begin{bmatrix} \beta \\ \alpha + \beta \end{bmatrix} \cdot e_c[k] \quad (3.7)$$

The Costas Loop implemented in GNU Radio consists of several key stages. First, the signal is rotated by the phase offset estimated in the previous stage. The phase detector obtains a new error metric to then begin the proportional-integral (PI) stage. Each stage is detailed now.

Phase Derotation

The signal flow commences with the discrete complex baseband input signal $x[n]$, which arrives at the receiver carrying an instantaneous phase perturbation. The primary operational stage of the diagram involves a complex multiplier (or mixer) that interacts the input signal with the corrective feedback phasor generated by the Numerically Controlled Oscillator (NCO).

```
1 | const gr_complex nco_out = gr_expj(-d_phase);
2 | gr::fast_cc_multiply(optr[i], iptr[i], nco_out);
```

Consequently, the phase-corrected output signal $y[n]$ is defined by the discrete equation:

$$y[n] = x[n] \cdot e^{-j\hat{\theta}[n-1]}$$

This operation rotates the sample according to the estimated phase deviation.

Nonlinear Phase Detection and Error Metric Extraction

Following the derotation, the corrected complex sequence $y[n] = \text{Re}\{y[n]\} + j \text{Im}\{y[n]\}$ is passed to the Phase Detector (PD) block, whose mathematical objective is to suppress the underlying data modulation and isolate the scalar angular deviation.

```
1 | return ((sample.real() > 0.0f ? 1.0f : -1.0f) * sample.imag() -
2 |         (sample.imag() > 0.0f ? 1.0f : -1.0f) * sample.real());
```

The result will be proportional to the sine of the phase difference between the vectors, providing a linear error metric for small angular deviations:

$$e[n] = \text{sgn}(\text{Re}\{y[n]\}) \cdot \text{Im}\{y[n]\} - \text{sgn}(\text{Im}\{y[n]\}) \cdot \text{Re}\{y[n]\} \quad (3.8)$$

On the other hand, under conditions where the receiver can estimate the signal-to-noise ratio (SNR), the algorithm transitions towards a Maximum Likelihood (ML) estimator over AWGN channels. In this regime, the sign operator is replaced by a hyperbolic tangent scaled by the instantaneous noise parameter γ :

$$e[k] = \tanh(\gamma \cdot I_{out}[k]) \cdot Q_{out}[k] - \tanh(\gamma \cdot Q_{out}[k]) \cdot I_{out}[k] \quad (3.9)$$

At low SNR ($\gamma \ll 1$), the Taylor approximation $\tanh(x) \approx x$ asymptotically reduces the discriminator to a classic fourth-power Costas loop, avoiding the amplification of thermal noise that a rigid hard-limiter would cause.

Furthermore, the algorithm imposes a strict symmetric hard-limiter on the error signal and keeps it strictly within the interval $[-1.0, 1.0]$.

```
1 | d_error = gr::branchless_clip(d_error, 1.0);
```

The Loop Filter

The bounded scalar error signal $e[n]$ is subsequently split into the parallel proportional and integral branches (PI filter) of the digital loop filter. This filter dictates the dynamic trajectory of the system by updating two internal state variables: the frequency estimate $f[n]$ and the phase estimate $\theta[n]$. The discrete-time difference equations governing this dynamic core are:

$$\hat{f}[k+1] = \hat{f}[k] + \beta \cdot e_c[k] \quad (3.10)$$

$$\hat{\theta}[k+1] = \hat{\theta}[k] + \hat{f}[k+1] + \alpha \cdot e_c[k] \quad (3.11)$$

where α and β are the proportional and accumulative gains, respectively.

```
1 | void advance_loop(float error)
2 | {
3 |     d_freq = d_freq + d_beta * error;
4 |     d_phase = d_phase + d_freq + d_alpha * error;
5 | }
```

The complete transfer function from the phase error $E(z)$ to the NCO phase $\Phi_{nco}(z)$ (see Figure 3.6 $\rightarrow$ p.15) is:

$$H_{loop}(z) = \frac{\Phi_{nco}(z)}{E(z)} = \underbrace{z^{-1}}_{\text{NCO Delay}} \cdot \underbrace{\frac{1}{1-z^{-1}}}_{\text{Phase Accum.}} \left(\alpha + \beta \cdot \underbrace{\frac{1}{1-z^{-1}}}_{\text{Freq. Accum.}} \right) \quad (3.12)$$

The aforementioned gains are calculated from the transfer function in the analog domain. PLL theory defines two key variables: the damping factor—which is fixed at $\sqrt{2}/2$ —and the bandwidth—the only configurable parameter—. To transition to the digital domain, a bilinear transform is performed to ensure the stability of the system. Then, the values of the parameters β and α are calculated as follows:

$$\alpha = \frac{4\zeta B_n}{1 + 2\zeta B_n + B_n^2} \quad (3.13)$$

$$\beta = \frac{4B_n^2}{1 + 2\zeta B_n + B_n^2} \quad (3.14)$$

For further details on the analog to digital domain and development of Equation (3.14) $\rightarrow$ p.16 and Equation (3.13) $\rightarrow$ p.16, refer to Chapters 2 & 13 [17].

3.4 Costas Loop Characterization

Once the mathematical model of the Costas Loop implemented in GNU Radio has been characterized, we will demonstrate its operational behavior through simulations. Understanding

the algorithm's performance boundaries in a practical scenario is crucial for properly configuring its parameters and delivering a functional solution.

As previously discussed, the loop must fulfill at least two key functions. First, it requires an acquisition range compatible with the scenarios proposed in [Figure 3.4 → p.13]. Second, given that the frequency varies rapidly over time — especially during the time of closest approach — the loop must be capable of tracking this frequency dynamically to maintain lock. To verify these requirements, we have conducted three experiments analyzing the following characteristics.

Experiment 1: Transient Response

To evaluate the loop's transient response, [Figure 3.15 → p.17] plots the step response of the system when subjected to an instantaneous 10 kHz frequency deviation. The convergence time depends directly on the configured loop bandwidth. A wider bandwidth allows the loop to react more aggressively to phase errors, significantly reducing the acquisition time for higher values of B_n .

Furthermore, the transient response time must be sufficiently brief to continuously track the maximum Doppler rate of 300 Hz/s (further analyzed in [Section 3.4 → p.18]). By selecting an appropriate bandwidth, and considering that frames have a length of 1115 bytes — assuming $I = 5$ —, this acquisition transition spans only a few tens to hundreds of bits. While a lack of synchronization over this duration might initially appear detrimental, it does not pose a significant operational issue for the overall link. This brief acquisition phase occurs exclusively during the initial signal lock-on period prior to data payload extraction. Any transient bit errors generated during this window are either absorbed by the Access Sync Marker (ASM) correlation search or inherently corrected by the concatenated FEC decoders once steady-state tracking is achieved. Moreover, in actual operation, the receiver tracks continuous kinematic curves rather than discontinuous 10 kHz steps, as previously illustrated in [Figure 3.5 → p.14].

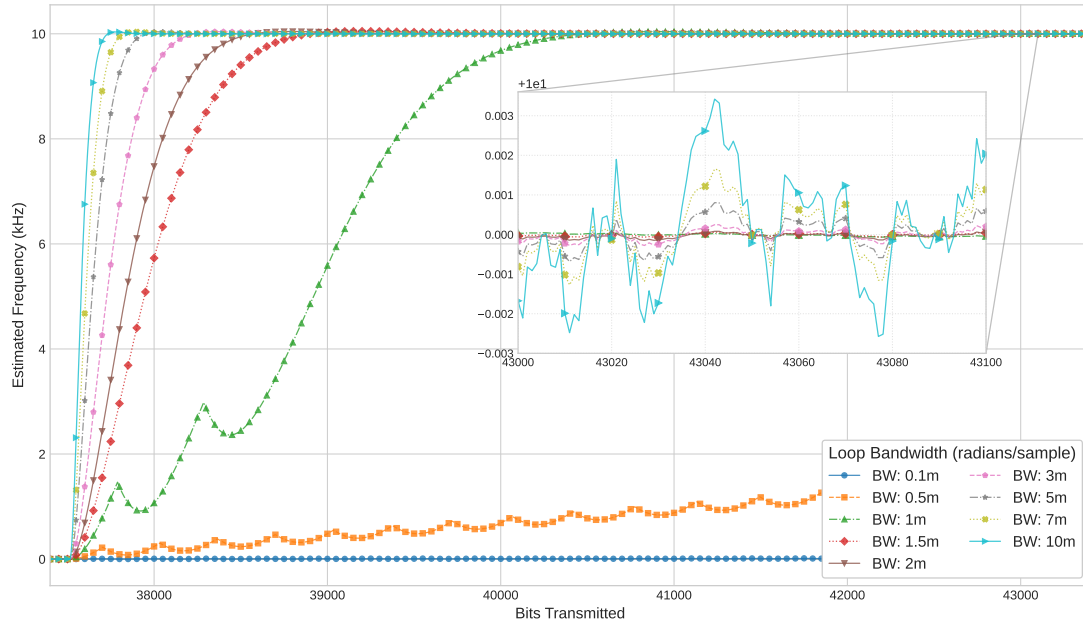


Figure 3.15 / Costas Loop frequency error transient response to a 10 kHz step across varying loop bandwidths.

Experiment 2: Frequency Acquisition Range

In digital receiver design, it is established in the literature that a pure Costas Loop typically loses lock when the initial frequency offset approaches approximately 5% of the symbol rate [17]. Given the system's transmission rate of 12.5 Mbaud, the algorithm is theoretically constrained

to a maximum frequency offset of around 0.625 MHz. This fundamental limitation is strictly corroborated by our empirical simulations and is largely independent of the configured loop bandwidth.

Recall [Section 3.3 → p.15], which details the phase detector mechanism. When the residual frequency error is large enough that the phase rotation between consecutive samples exceeds the detector's decision boundary (for instance, $\pm 45^\circ$ in QPSK), ambiguity or aliasing occurs within the constellation. The received symbol crosses into an adjacent quadrant, misleading the decision-directed phase detector algorithm. The detector assumes an incorrect target symbol and consequently calculates an error metric with an inverted sign. As a result, the feedback loop instructs the NCO to correct in the diametrically opposite direction, pushing the system further away from the true carrier frequency. This immediately triggers a cycle slip and results in a total loss of synchronization.

Experiment 3: Frequency Tracking

Finally, the Costas Loop must be capable of tracking a frequency transition of at least 300 Hz/s, as established in [Figure 3.4 → p.13]. To evaluate this, an oscillatory perturbation is introduced to simulate a dynamic frequency offset. As depicted in [Figure 3.16 → p.18], the frequency shifts linearly (refer to the lower subplot displaying the derivative), allowing us to identify the exact point at which the loop loses lock.

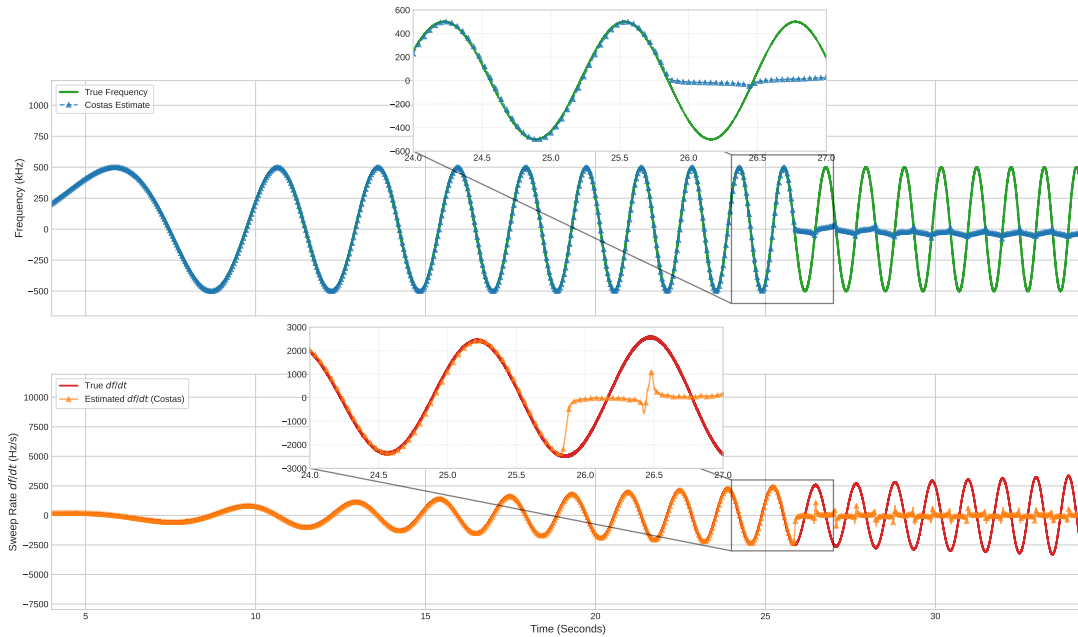


Figure 3.16 / Frequency tracking performance: True vs. Estimated frequency. The Costas Loop loses lock at a sweep rate of 2.5 kHz/s for a simulated loop bandwidth of $B_n = 5$ mrad/s.

Consequently, evaluating this behavior across different bandwidths yields the experimental results shown in [Figure 3.17 → p.19]. This figure is highly valuable as it enables the precise selection of the required bandwidth B_n to configure the Costas Loop. According to the theoretical tracking limits defined by [17], the maximum sweep rate that can be tracked is $\Delta\dot{\omega} \approx \omega_n^2/2$, which closely matches our empirical findings. This demonstrates that if our system must track a maximum drift of 300 Hz/s (see [Figure 3.4 → p.13]), we need to configure a loop bandwidth of at least ≈ 2 mrad/s. This establishes our stability region.

Returning to [Figure 3.15 → p.17], an examination of the magnified plot reveals that the frequency estimation exhibits oscillations that become more pronounced with wider configured bandwidths. There is a steep penalty for exceeding the optimal bandwidth, as extensively discussed in

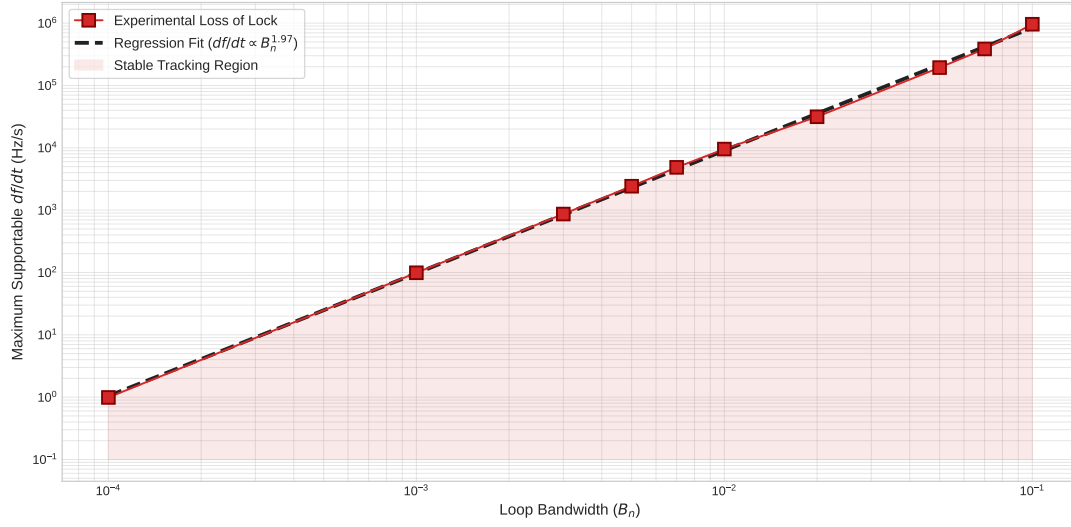


Figure 3.17 / Costas Loop Robustness: Maximum trackable frequency sweep rate (df/dt) versus Loop Bandwidth.

[Chapter 6 \[17\]](#). Specifically, the relationship between the loop bandwidth and the phase variance of the NCO is governed by the equation:

$$\sigma_{\theta_{no}}^2 = \frac{2N_0 B_L}{V_s^2} = \frac{W_0 B_L}{P_s} \text{ [rad}^2\text{]} \quad (3.18)$$

The noise generated by the estimation process is superposed onto the data signal, causing a considerable degradation in link performance. This is clearly observed in [Figure 3.19 \$\rightarrow\$ p.20](#). Because deviating from the optimal normalized bandwidth incurs a heavy cost, we recommend a value of at least 1 to 2 mrad/s to ensure the proper functioning of the link. According to the referenced figure, this operating point corresponds to the cyan curve.

It must be noted that initial system simulations under these bandwidth configurations exhibited a higher than expected Frame Error Rate (FER), resulting in a significant number of dropped frames, as represented by the baseline pink curve shown in [Figure 4.1 \$\rightarrow\$ p.21](#). This severe performance degradation is not a failure of the Costas Loop's tracking ability, but rather a fundamental consequence of unhandled phase ambiguities. While the loop successfully recovers the carrier frequency and achieves phase lock, decision-directed algorithms inherently exhibit an N -fold phase ambiguity (e.g., 90° phase rotations in QPSK). If the loop locks onto an incorrect constellation quadrant, the demodulated bitstream is systematically inverted or rotated, rendering the data completely undecodable by the subsequent Forward Error Correction (FEC) blocks. To recover the true theoretical BER performance, the system requires a mechanism to resolve this ambiguity and detect the exact boundaries of the data structures. This critical requirement, and its specific algorithmic implementation, are addressed in the following chapter regarding Frame Synchronization ([Chapter 4 \$\rightarrow\$ p.21](#)).

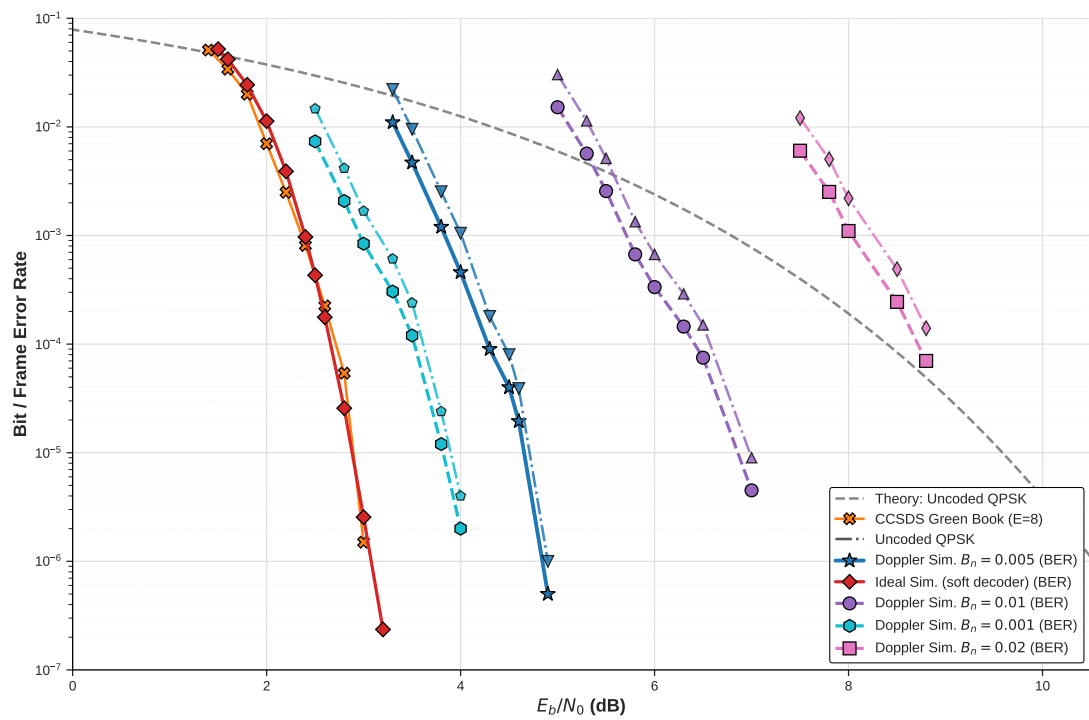


Figure 3.19 / Comparative Bit Error Rate (BER) performance across different normalized loop bandwidths (B_n).

4 Frame Synchronization

Frame synchronization is a well-known problem in the literature. We receive several frames with a defined length and a preamble (ASM) designed to locate them. To search for the preamble within the received samples and thus locate each frame, there are two techniques mentioned in the standard [8]:

First, an intuitive approach would be to search for the decoded preamble using the **Viterbi** algorithm. Since the ASM is a pseudorandom code specified in [1], it could be correlated¹ with the preamble until it is found. However, we face the problem of the phase ambiguity of the Costas loop; the constellation after the block can appear rotated.

Secondly, it is possible to correlate with the preamble before synchronization and the **Viterbi** decoder by searching for the sequence encoded by the convolutional code. Although the output of the convolutional code depends on the previous K bits, and assuming that those bits are random, once we bypass those bits, the encoder output will yield the same result if the input is the same. If the ASM preamble measures 32 bits, then the preamble encoded with a rate 1/2 CC will measure 64 bits, of which 50 bits will be identical² since $K = 7$ (Remember Section 1.3 $\rightarrow$ p.3).

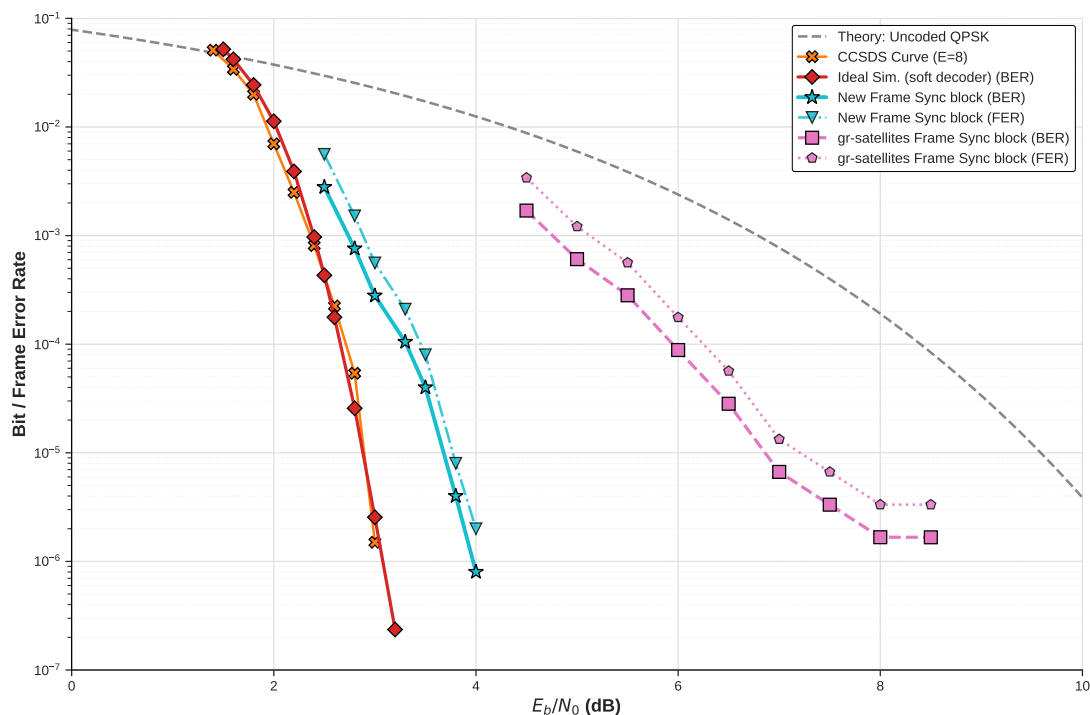


Figure 4.1 / Performance comparison between the standard frame synchronization algorithm provided by the `gr-satellites` library and a custom-implemented synchronizer.

Although it may seem that searching for 50 bits is better than searching for 32 — even though the coding gain is a factor to consider —, those bits are being searched for without having fine frequency synchronization. Therefore, a search based on some differential scheme is necessary. An advantage offered here is resolving the phase ambiguity problem, since the GNU Radio

¹Although there are other methods to find a known bit string, the correlation method has been chosen and is widely used in the literature.

²As a curiosity, the standard is very rigorous regarding spurious signals and power peaks at certain frequencies. By having 50 repeating bits, spectral peaks will be generated at a distance of $\text{frame_length} / (\text{frame_length} + \text{ASM}) \cdot T_s$. Since these bits have a very low power compared to the other bits that make up the frame, this is considered permissible.

Costas loop implementation accepts a tag indicating the relative phase, which can be obtained by correlating.

In this report, we have focused on implementing the first solution.

4.1 Algorithms Already Implemented in GNU Radio

Several blocks that implement very similar algorithms to solve this problem can be found online. In general, they all focus on correlating with the preamble, and if the result of the correlation is greater than an established value (threshold), then a metadata tag is added and it is considered that the preamble (and consequently the frame) has been found. In the GNU Radio implementations **Correlation Estimator** and in the gr-satellites library **CCSDS Deframer**, the correlation is performed against the entire incoming bit block, generating false negatives and false positives, which drastically reduces the viability of the link at low SNR.

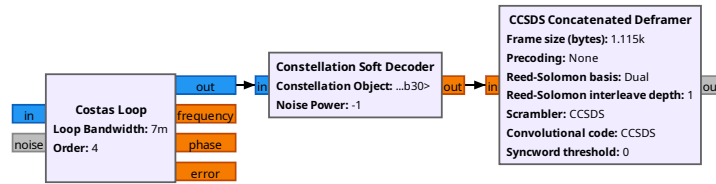


Figure 4.2 / Receiver architecture with the deframer block from [3] library.

To understand the limits of using an Access Sync Marker (ASM) of length $N = 32$ with an error threshold $T = 3$ (we tolerate up to 3 bits of difference), we must separate the channel's physical Bit Error Rate (p) from the correlator's False Alarm Probability (P_{FA}). While $T = 3$ ensures robust synchronization in noisy channels (tolerating up to $p \leq 0.026$), continuously correlating the entire incoming bitstream introduces a major issue: false positives.

Assuming the payload or ambient noise acts as random data ($p_{rand} = 0.5$), the probability of a random sequence falsely matching the ASM within our 3-bit tolerance is:

$$P_{FA} = \sum_{k=0}^3 \binom{32}{k} (0.5)^{32} \approx 1.28 \times 10^{-6} \quad (4.3)$$

Consequently, $T = 3$ creates an artificial error floor. Even if the radio link is perfect (infinite SNR with zero physical errors), the receiver will still trigger a false sync approximately once every 7.8×10^5 bits. Since the system's effective error rate can never drop below this 1.28×10^{-6} limit, relying on continuous brute-force correlation is suboptimal. You can see in [Figure 4.1 $\rightarrow$ p.21] the fictitious floor in the pink curve. The algorithm must be improved, for example, by implementing a state machine that only opens correlation windows where the next preamble is actually expected, as the standard says.

Phase Ambiguity

In fact, the phase ambiguity problem is not completely resolved. Since QPSK is used, we have an unknown phase at the output of the Costas loop. The constellation may have rotated 0 , $\pi/2$, π , or $3\pi/2$ radians depending on which quadrant the Costas loop managed to lock onto after an assumed loss of lock.

On the other hand, we have a **Viterbi** decoder which is transparent, meaning that if the input to the **Viterbi** is inverted (assume the case of a π radians ambiguity), then the output will also be inverted. This allows applying the algorithms by searching for the phase 0 ASM and the VSN rotated by π radians. This is, in fact, what is already implemented in gr-satellites **Concatenated Deframer**. However, what happens when the phase ambiguity is $\pi/2$ or $3\pi/2$? The **Viterbi** decoder is unable to decode and outputs garbage. This latter issue is not addressed

in the found implementations since they are mainly based on BPSK (therefore, the constellation can only be rotated by π radians, and since the decoder is transparent, there is no problem).

Finally, the standard [Section 9.3.7 \[8\]](#) recommends applying a multi-state flywheel algorithm. In this way, once a frame is found ("search" state), the algorithm transitions to a "lock" state and considers that there are up to a configurable number N of frames in the space adjacent to the found frame, even if the ASM has not been found. This allows recovering the frame even if the preamble has been destroyed by an error burst and we solve the problem of false positives that we were talking about.

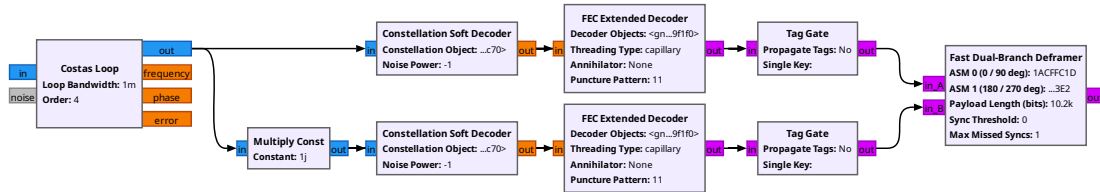


Figure 4.4 / Dual-branch receiver architecture with parallel Viterbi decoding and fast synchronization.

4.2 Solving the Phase Problem

Having outlined the phase ambiguity problem introduced by the Costas loop (which can lock at 0 , $\pi/2$, π , or $3\pi/2$ radians) and the limitations of existing implementations, it is necessary to implement a custom, robust synchronization algorithm.

While a brute-force approach running four parallel decoding pipelines is computationally feasible on modern hardware, a more resource-efficient trade-off can be achieved. Because the Viterbi decoder is transparent to π -radian phase inversions, the ambiguity can be fully resolved using only two parallel branches. The first branch processes the incoming soft symbols directly (covering the 0 and π shifts), while the second branch processes a $\pi/2$ -rotated version of the symbols (covering the $\pi/2$ and $3\pi/2$ shifts).

An initial feedback-based approach using asynchronous messages to rotate the Costas loop output was evaluated. However, this design was discarded because buffer latencies caused a systematic loss of frames during phase slips (a detailed analysis of this discarded architecture and its state machine is provided in [C](#)).

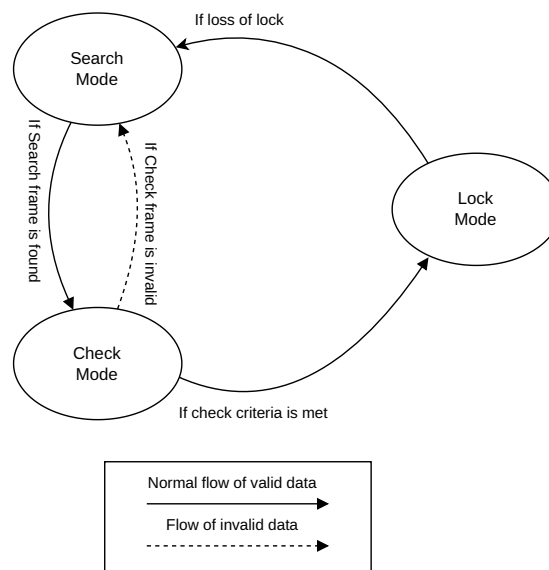


Figure 4.5 / Three-state diagram (Search/Lock) for the dual-branch frame synchronizer.

To achieve zero-latency recovery and eliminate frame loss, the custom dual-branch synchronizer employs a highly optimized three-state machine (see [Figure 4.5](#) → p.23) that operates continuously on both data streams:

- **Search mode:** The 32-bit ASM and its inverted version are simultaneously searched across both the standard and the $\pi/2$ -rotated branches. Upon detecting a candidate synchronization marker within the acceptable bit-error threshold, the algorithm transitions to *Check* mode.
- **Check mode:** In this intermediate verification state, the algorithm evaluates the synchronization criteria to ensure the detected marker is not a false positive generated by random payload data. It confirms the validity of the sequence, definitively selects the active branch containing the valid data stream, and transitions to *Lock* mode.
- **Lock mode:** The search window systematically jumps forward by the exact frame length to evaluate the subsequent ASM on the active branch (flywheel operation). If the ASM is missing or corrupted beyond the threshold, a `max_misses` counter is incremented. If this counter overflows—indicating a genuine loss of synchronization or a Costas loop phase slip—the system instantly drops the current branch and returns to *Search* mode. Because the alternative branch is already being decoded in parallel, the synchronizer can re-establish lock on a new phase almost instantaneously.

4.3 Results

By leveraging this dual-branch feed-forward architecture, the receiver maintains precise synchronization regardless of the Costas loop's phase orientation. Since both the non-rotated and $\pi/2$ -rotated symbols are continuously fed into parallel **Viterbi** decoders, a phase slip simply triggers a rapid transition from *Lock* to *Search* mode, where the algorithm instantly latches onto the alternative branch.

The improvements in link viability are clearly demonstrated in [Figure 4.1](#) → p.21. The Bit / Frame Error Rate is plotted against the E_b/N_0 (dB). As previously theorized, the continuous brute-force correlation used by the gr-satellites Frame Sync block (BER) and the gr-satellites Frame Sync block (FER) creates an artificial error floor, preventing the error rate from dropping below the 10^{-6} limit. In contrast, the New Frame Sync block (BER) closely tracks the Ideal Sim. (soft decoder) (BER) curve without exhibiting any artificial floor. By actively gating the correlation windows through the three-state machine, the new synchronizer avoids false alarms from payload data or ambient noise, ensuring near-ideal performance.

5 Link Budget and Final Results

5.1 Link Budget Analysis

To validate the feasibility of the high-rate X-band downlink, a comprehensive link budget analysis was conducted. The theoretical framework and parameter estimation for this budget rely on orbital data extracted from Systems Tool Kit (STK) combined with standard AMSAT/IARU link calculation models. While certain secondary parameters with negligible impact on the final path loss were omitted to streamline the analysis, the resulting budget is robust and accurately reflects the physical constraints of the mission.

Item	Symbol	Adverse	Nominal	Favourable	Dimension
Frequency	f	10.475	10.475	10.475	GHz
Data Rate	R	10	25	25	Mbps
S/C Output Power	P	0.47	1.48	5.29	W
S/C antenna Gain	G_t	17	19	21	dBi
S/C antenna HPBW	θ_t	20.2	17	15.3	deg
S/C pointing error	–	± 7.5	± 5	± 3	deg
S/C pointing loss	L_t	-1.7	-1.04	-0.46	dB
Line loss	$L_{\theta,t}$	-0.6	-0.5	-0.4	dB
EIRP	–	9.27	17	25.2	dBW
S/C max. altitude	H	505	500	495	km
Atmospheric Loss	L_a	-1.9	-2.1	-2.2	dB
GS min. elevation	ϵ	15	15	15	deg
GS antenna Gain	G_r	40	40.7	43.1	dBi
Polarization loss	L_{pol}	-0.5	-0.3	-0.1	dB
GS antenna HPBW	θ_r	2.8	1.4	0.8	deg
GS Pointing error	–	± 1	± 0.5	± 0.25	deg
GS pointing loss	$L_{\theta,r}$	-1.53	-1.53	-1.17	dB
GS noise T°	T_0	148.5	135	121.5	K
GS FoM	G_r/T	15.2	15.7	16.1	dB/K
Implementation losses	L_i	-2	-2	-2	dB
E_b/N_0	–	4.1	7.1	18.6	dB
Required E_b/N_0	–	2.3	2.3	2.3	dB
Margin	–	1.8	4.8	16.3	dB

Table 5.1 / Payload Downlink budget for the CHES mission.

The calculation evaluates three distinct operational scenarios: Favourable, Nominal, and Adverse. As detailed in (Table 5.1 $\rightarrow$ p.25), the nominal scenario yields a link margin of 4.8 dB. According to the European Cooperation for Space Standardization (ECSS) guidelines and classical satellite communication literature [9], a minimum margin of 3 dB is highly recommended for nominal LEO downlinks to absorb unmodeled atmospheric attenuation or transient pointing errors. Therefore, our nominal configuration safely complies with aerospace standards. In the worst-case (Adverse) scenario, the margin is reduced to 1.8 dB. This extremely tight tolerance highlights that employing the concatenated Convolutional and Reed-Solomon coding scheme (Section 1.1 $\rightarrow$ p.1) is not merely an option, but a strict requirement to maximize coding gain and successfully close the link under degraded conditions.

Furthermore, the budget deliberately assumes a conservative 2 dB implementation loss (L_i). However, empirical evaluation of our Software-Defined Radio architecture demonstrates that the actual penalty is slightly lower. As illustrated by the performance evaluation in [Figure 5.3](#), the discrepancy between the theoretical standard curve and our practical software implementation is approximately 1.4 dB. This confirms that the software receiver performs well within the established budget boundaries.

5.2 Final Results and Real-Time Execution

A central objective of this project was to verify whether standard C++ based DSP blocks in GNU Radio could sustain the mission’s high-throughput requirements without relying on dedicated hardware acceleration, such as FPGAs. The transition from application-specific integrated circuits to algorithms running on Commercial Off-The-Shelf (COTS) servers introduces heavy computational constraints.

To test the system’s operational limits, the complete receiver pipeline was deployed on a compact base station server ([Geekom A8 Max](#), featuring an AMD Radeon 780M architecture and 32 GB of RAM). Under these real-world execution conditions, the software receiver successfully sustained a stable, real-time decoding throughput of 28 Mbps without encountering buffer overflows or frame drops. This result comfortably exceeds the nominal target of 25 Mbps specified in [Table 5.1](#), conclusively demonstrating that a purely software-defined architecture is fully viable for this CHESS mission scenario.

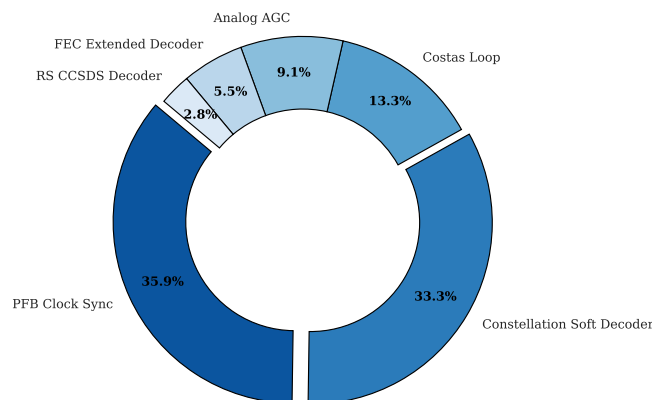


Figure 5.2 / Relative CPU resource allocation among the primary DSP modules of the receiver pipeline.

To further understand the computational bottlenecks of the system, a performance profiling analysis was conducted during the real-time execution. [Figure 5.2](#) illustrates the relative CPU consumption of the most intensive signal processing blocks within the flowgraph. The remaining modules possess a negligible computational footprint. As depicted, the tracking and decoding stages — specifically the Costas Loop, the Polyphase Clock Sync, and the Constellation Soft Decoder — monopolize the vast majority of the system’s resources. This performance distribution clearly demonstrates that if future mission profiles demand an increase in the data rate beyond 28 Mbps, optimization efforts (such as SIMD vectorization or multi-threading) must be strictly targeted at these specific synchronization blocks.

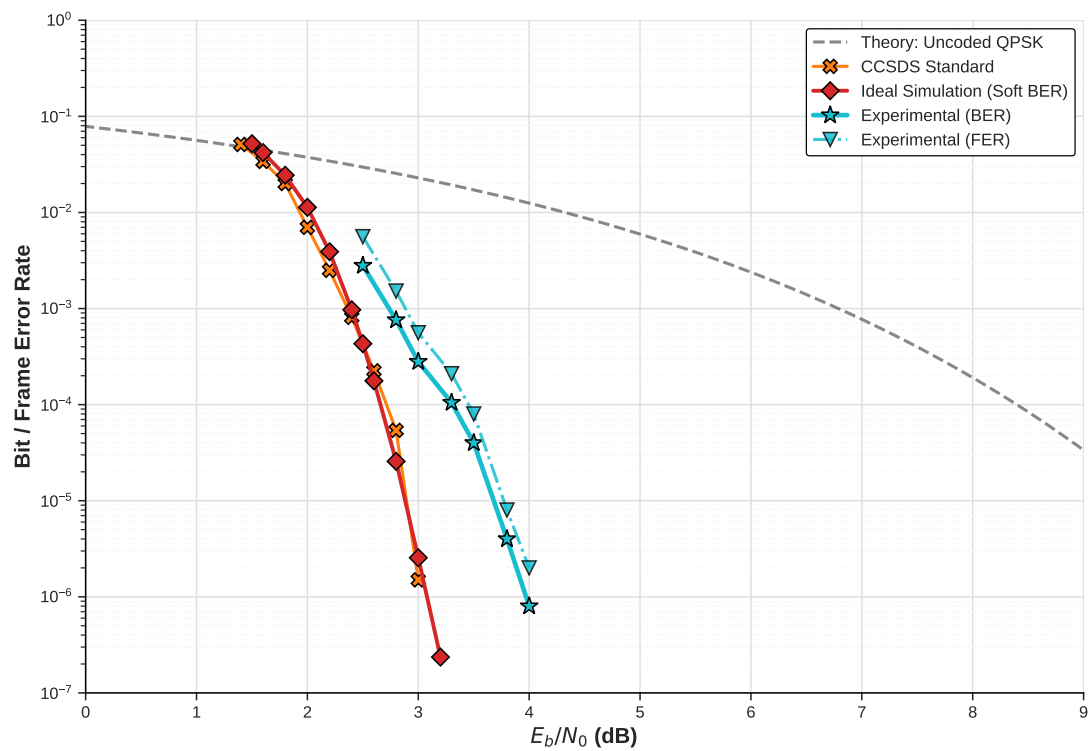


Figure 5.3 / Final bit and frame error rate performance of the complete SDR pipeline. The experimental curve illustrates the real-time execution including Doppler tracking and dual-branch synchronization, compared against the ideal simulated soft-decoding performance, the theoretical CCSDS limit, and the uncoded QPSK baseline.

A Case Study on QO-100 Narrowband Beacon

6.1 Ground Station Hardware Architecture

The ground segment for high-rate data acquisition and capture is structured as a sequential RF chain. The architecture is designed to maintain signal integrity from the antenna focal point to the digital processing unit, ensuring that the noise figure is dominated by the LNB's front-end stage to support the mission's link margin requirements.

Antenna → LNB → Bias-T → SDR → Computer

The first block consists of the radiating system, composed of a parabolic antenna whose feed is mechanically fixed at the primary focus using a rigid four-arm support structure or quadripod. Directly coupled at this focal point is the low-noise block downconverter, or LNB. The critical function of the LNB is to amplify the extremely weak signal received from outer space (around X-band frequencies or 10 GHz) while injecting the lowest possible amount of thermal noise, and then translate it in frequency using a local oscillator coupled to a significantly lower Intermediate Frequency (IF) (typically in L-band, between 1 and 2 GHz). This downconversion is essential so that the signal can propagate through an extended-length coaxial cable to the equipment room without suffering severe attenuation or distortion due to high-frequency dielectric losses.

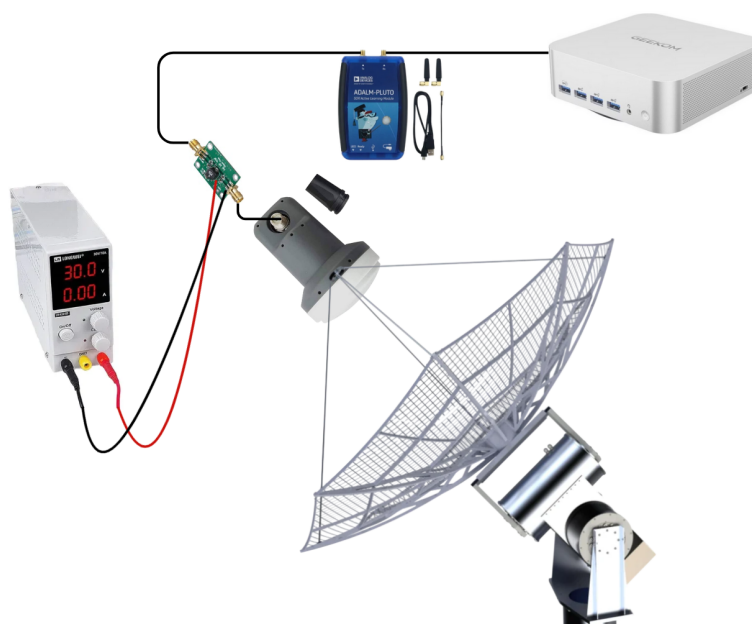


Figure 6.1 / Experimental hardware setup configured for the QO-100 geostationary satellite reception test.

Downstream in the transmission line, the IF-modulated signal intercepted by the coaxial cable passes through a bias tee or Bias-T. This device acts as a direct current (DC) and RF diplexer, allowing the necessary supply voltage to be injected to activate the LNB's active circuits through the coaxial cable core itself, while simultaneously isolating the high-frequency signal component to protect subsequent stages. Subsequently, the filtered analog stream enters the software-defined radio or SDR, which is responsible for performing complementary analog filtering, automatic

gain control, and analog-to-digital conversion (ADC) to generate the in-phase and quadrature (I/Q) digital samples. Finally, these digitized samples are transmitted via a high-speed USB bus interface to the host computer (Computer), where the previously analyzed digital signal processing, frame synchronization, and channel decoding pipeline is executed in real time.

6.2 The Es'hail-2 Satellite and Multimedia Beacon

To physically validate the RF front-end architecture described in the previous section, live satellite reception tests were required. Currently, the 2.4-meter ground station antenna lacks an active mechanical tracking system, rendering the capture of fast-moving Low Earth Orbit (LEO) satellites operationally unfeasible for the time being. To circumvent this physical limitation, we targeted **Es'hail-2** (also known as QO-100), a geostationary satellite positioned statically at 26° East. Because it remains fixed relative to the ground station, it serves as an ideal geometric target to verify the integrity of our hardware chain.



Figure 6.2 / Hardware setup deployed for the QO-100 reception test, showcasing the SDR and the connected RF chain.

Es'hail-2 hosts the first geostationary amateur radio transponder, developed by **AMSAT-DL**. Specifically, the payload includes a **narrowband (NB) transponder** with a downlink spanning from 10489.550 to 10489.800 MHz. Within this band, the satellite continuously broadcasts a multimedia beacon.

It is crucial to note that this multimedia beacon operates using the **DVB-S2** (Digital Video Broadcasting - Satellite - Second Generation) standard, which relies on LDPC and BCH forward error correction. This contrasts fundamentally with the CCSDS (Reed-Solomon and Convolutional Coding) protocol developed throughout this project for the **CHES** mission. Because the coding schemes are mutually incompatible, our custom GNU Radio receiver pipeline

cannot be used to decode the QO-100 beacon. Consequently, this chapter serves as a valuable bonus milestone. Rather than testing our specific software demodulator blocks, it focuses on validating the physical hardware and the RF reception capabilities of the ground segment under real-world operating conditions. More importantly, it provides an ideal testbed to verify the execution of an alternative GNU Radio script in a live scenario, demonstrating the framework's capability to stream data, handle hardware interfaces, and perform complex baseband demodulation in real time.

6.3 Hardware Validation and Demodulation Results

The complete reception system was assembled and pointed towards the 26° East orbital slot. The hardware setup, including the antenna feed, the SDR, and the data acquisition laptop, is depicted in [Figure 6.2](#) → p.29.

Since our primary pipeline was strictly designed for CCSDS telemetry, we relied on the open-source community to perform the DVB-S2 software demodulation and validate our baseband I/Q samples. To achieve this, we utilized an existing, highly optimized GNU Radio flowgraph developed by Daniel Estévez [\[18\]](#) [\[19\]](#). His implementation handles the DVB-S2 physical layer framing, symbol synchronization, and LDPC decoding required to extract the payload.

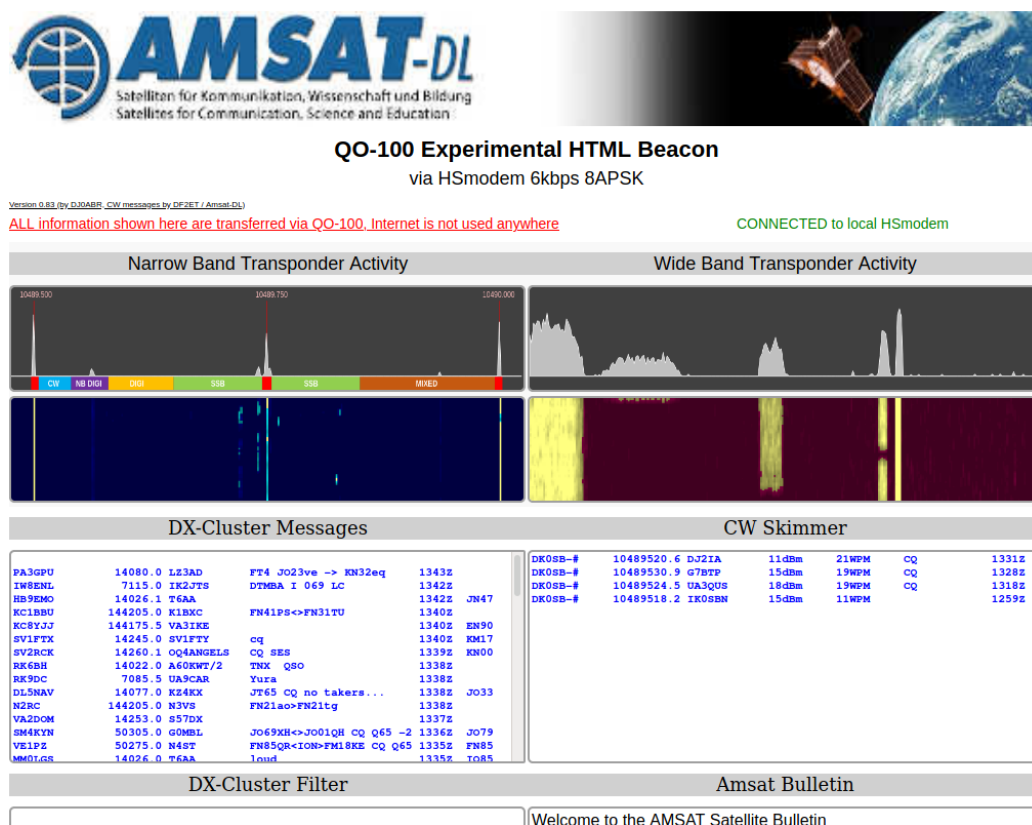


Figure 6.3 / Demodulated HTML dashboard extracted from the QO-100 multimedia beacon, displaying live telemetry and transponder status.

The test was highly successful. The RF chain captured the X-band signal cleanly, and the third-party SDR flowgraph locked onto the beacon seamlessly. The resulting demodulated data stream from the DVB-S2 beacon consists of a periodically broadcasted HTML file. As shown in [Figure 6.3](#) → p.30, this file renders a multimedia dashboard containing real-time satellite telemetry, transponder operational status, and AMSAT-DL bulletins. Successfully recovering this digital file provides definitive proof that the ground station's hardware architecture, from the parabolic

reflector down to the USB bus, introduces minimal noise and operates perfectly within the required X-band specifications.

Physical Layer Error Probability

A.1 Optimal Detection for the Vector AWGN Channel

Consider a digital communication system transmitting information over a continuous time physical medium. The information source generates a message m chosen from a finite discrete alphabet $\mathcal{M} = \{1, 2, \dots, M\}$. The transmitter maps this message into a distinct continuous-time signal waveform $s_m(t)$ with finite energy. As this signal propagates through the Additive White Gaussian Noise (AWGN) channel, it is corrupted by a zero-mean random noise process $n(t)$.

$$\mathbf{r} = \mathbf{s}_m + \mathbf{n}$$

The fundamental objective of the receiver is to employ a decision operation, a mapping $D : \mathbb{R}^N \rightarrow \mathcal{M}$, that observes $\mathbf{r}$ and produces an estimate $\hat{m}$ of the transmitted message. To minimize the probability of making an erroneous decision, the receiver must implement the Maximum A Posteriori (MAP) detection criterion. Applying Bayes' theorem, and letting P_m denote the prior probability of transmitting message m , the decision rule is formulated as:

$$\begin{aligned} \hat{m} &= \arg \max_{m \in \mathcal{M}} \{P_m p(\mathbf{r}|\mathbf{s}_m)\} \\ &= \arg \max_{m \in \mathcal{M}} \left\{ P_m \left(\frac{1}{\sqrt{\pi N_0}} \right)^N \exp \left(- \frac{\|\mathbf{r} - \mathbf{s}_m\|^2}{N_0} \right) \right\} \end{aligned}$$

where $N_0/2$ represents the two-sided power spectral density of the noise, and $p(\mathbf{r}|\mathbf{s}_m)$ is the multivariate Gaussian conditional probability density function.

In modern communication systems, the input data stream is typically passed through a randomizer or scrambler, rendering the transmitted signals equiprobable (i.e., $P_m = 1/M$ for all $m \in \mathcal{M}$). Under this condition, the MAP detector systematically simplifies into the Maximum Likelihood (ML) detector. Since the exponential function is monotonically decreasing, maximizing the likelihood is mathematically equivalent to minimizing the squared Euclidean distance between the received vector $\mathbf{r}$ and the candidate signal points $\mathbf{s}_m$. Therefore, the optimal decision rule simplifies to the minimum-distance detector:

$$\begin{aligned} \hat{m} &= \arg \max_{m \in \mathcal{M}} \{p(\mathbf{r}|\mathbf{s}_m)\} \\ &= \arg \min_{m \in \mathcal{M}} \|\mathbf{r} - \mathbf{s}_m\| \end{aligned}$$

This geometric interpretation directly dictates the formation of decision regions within the signal space $\mathbb{R}^N$. For any given signal point $\mathbf{s}_m$, its corresponding decision region $\mathcal{D}_m$ encompasses all points in the vector space that are closer to $\mathbf{s}_m$ than to any other candidate signal $\mathbf{s}_{m'}$. The boundary separating any two adjacent regions $\mathcal{D}_m$ and $\mathcal{D}_{m'}$ is a hyperplane comprising points strictly equidistant from both signals. Formally, the decision region is defined as the set:

$$\mathcal{D}_m = \{\mathbf{r} \in \mathbb{R}^N : \|\mathbf{r} - \mathbf{s}_m\| < \|\mathbf{r} - \mathbf{s}_{m'}\| \quad \forall m' \in \mathcal{M} \setminus \{m\}\}$$

To rigorously evaluate the performance of this system, we analyze the probability of symbol error, denoted as P_e . An error occurs whenever the received vector $\mathbf{r}$ falls outside the correct decision region $\mathcal{D}_m$ given that $\mathbf{s}_m$ was transmitted. By applying the law of total probability over the entire message alphabet $\mathcal{M}$, we obtain:

$$\begin{aligned} P_e &= \sum_{m \in \mathcal{M}} P_m P[\mathbf{r} \notin \mathcal{D}_m | \mathbf{s}_m] \\ &= \sum_{m \in \mathcal{M}} P_m P_{e|m} \end{aligned}$$

The conditional probability of error, $P_{e|m}$, is evaluated by integrating the conditional likelihood function over the complement of the decision region, $\mathcal{D}_m^c$, which is geometrically equivalent to summing the integrals over all mutually exclusive incorrect regions $\mathcal{D}_{m'}$:

$$\begin{aligned} P_{e|m} &= \int_{\mathcal{D}_m^c} p(\mathbf{r}|\mathbf{s}_m) d\mathbf{r} \\ &= \sum_{m' \neq m} \int_{\mathcal{D}_{m'}} p(\mathbf{r}|\mathbf{s}_m) d\mathbf{r} \end{aligned}$$

A.2 Error Probability for QPSK Modulation

We can now apply this theoretical framework to evaluate the Quadrature Phase Shift Keying (QPSK) scheme, where the cardinality of our alphabet is $|\mathcal{M}| = 4$. Due to the symmetric topology of the constellation, it is mathematically more tractable to compute the probability of a correct decision, $P_{c|m}^{\text{QPSK}}$, and subtract it from unity.

Because the noise processes mapped to the orthogonal basis functions are statistically independent, the two-dimensional integral over the defined region $\mathcal{D}_m$ factors perfectly into the product of two one-dimensional Gaussian integrals. Expressing this in terms of the standard Q-function and the minimum Euclidean distance $d_{\min}$ between adjacent constellation points, we find:

$$\begin{aligned} P_{e|m}^{\text{QPSK}} &= 1 - P_{c|m}^{\text{QPSK}} \\ &= 1 - \int_{\mathcal{D}_m} p(\mathbf{r}|\mathbf{s}_m) d\mathbf{r} \\ &= 1 - \left[1 - Q\left(\frac{d_{\min}}{\sqrt{2N_0}}\right) \right] \left[1 - Q\left(\frac{d_{\min}}{\sqrt{2N_0}}\right) \right] \\ &= 2Q\left(\frac{d_{\min}}{\sqrt{2N_0}}\right) - Q^2\left(\frac{d_{\min}}{\sqrt{2N_0}}\right) \end{aligned}$$

Because all symbols are assumed to be equiprobable and the constellation maintains strict rotational symmetry, the conditional probability of error is identical for every transmission. Consequently, the average symbol error probability for the entire QPSK system precisely mirrors the conditional probability:

$$\begin{aligned} P_e^{\text{QPSK}} &= P_{e|m}^{\text{QPSK}} \\ &= 2Q\left(\frac{d_{\min}}{\sqrt{2N_0}}\right) - Q^2\left(\frac{d_{\min}}{\sqrt{2N_0}}\right) \end{aligned}$$

Block Diagrams

This appendix provides a detailed visual representation of the core structural architectures analyzed throughout this project. It includes the comprehensive end-to-end communication block diagram implemented in GNU Radio, highlighting both custom and modified modules, as well as an illustrative example of the rectangular block interleaving matrix performance.

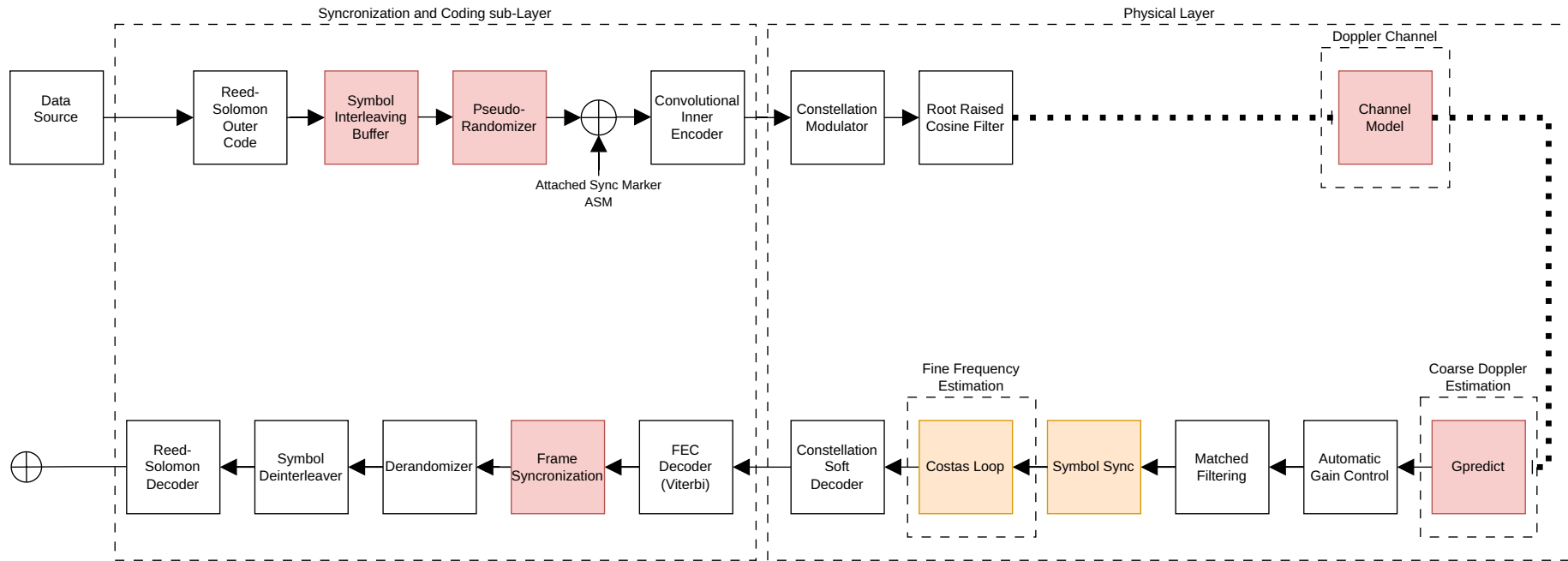


Figure B.1 / Complete block diagram of the end-to-end communication system implemented in GNU Radio. Red blocks represent custom DSP modules developed specifically for this project and orange blocks indicate existing library modules that were modified.

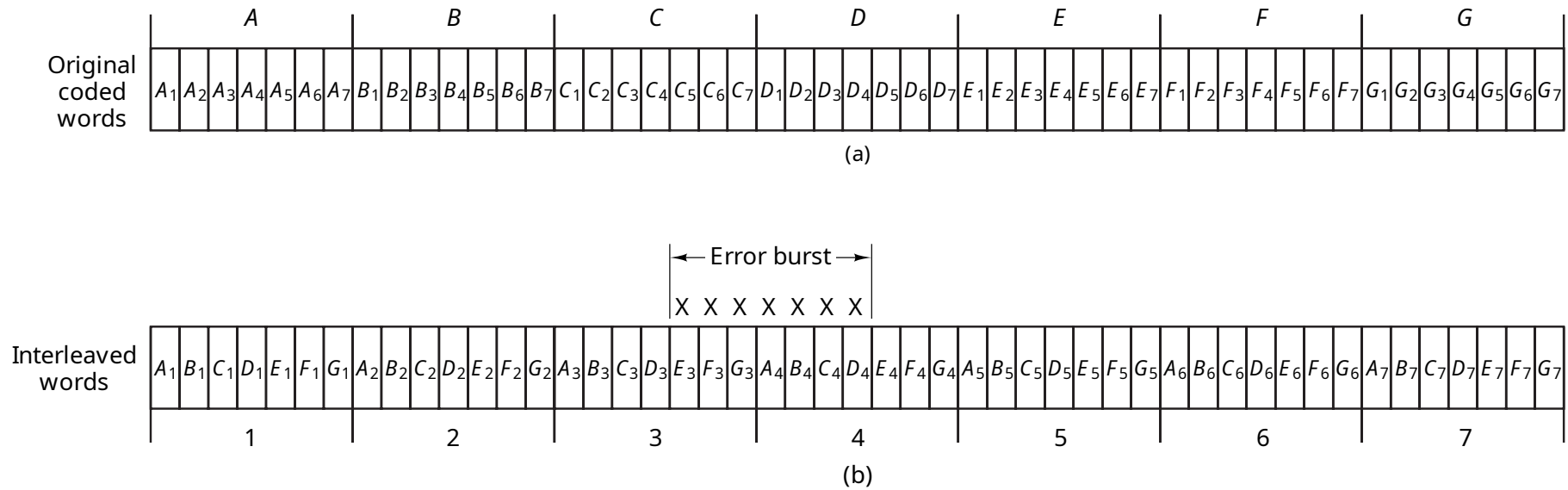


Figure B.2 / Interleaving example. (a) Original uninterleaved codewords, each comprised of seven code symbols. (b) Interleaved code symbols.

Alternative Synchronization Architectures

C.1 Single-Branch Asynchronous Frame Synchronizer

To resolve the phase ambiguity introduced by the Costas loop without the computational overhead of running multiple parallel **Viterbi** decoders, an asynchronous feedback-based frame synchronization algorithm was initially developed and evaluated.

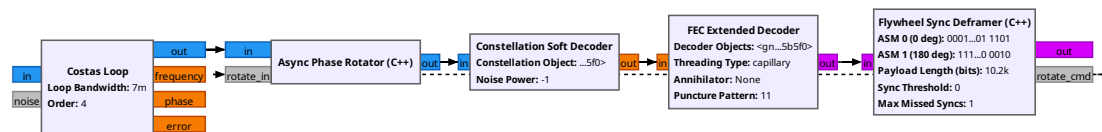


Figure C.1 / Receiver architecture with a custom single-branch flywheel algorithm.

This alternative architecture prioritizes computational speed and resource efficiency by utilizing only a single **Viterbi** decoding pipeline. Because the **Viterbi** decoder is transparent to π -radian phase inversions, the synchronizer can search for both the non-rotated Access Sync Marker (ASM) and its π -radian inverted counterpart simultaneously. To account for the remaining $\pi/2$ and $3\pi/2$ ambiguities, the algorithm utilizes a feedback loop: if the ASM is not found, it assumes the Costas loop has locked onto an orthogonal phase and sends an asynchronous message to a rotation block to shift the incoming constellation by 90° .

The logic of this algorithm relies on a four-state machine, illustrated in [Figure C.2 → p.38](#):

- **Search mode:** The 32-bit preamble (both normal and inverted) is searched for within the incoming bitstream. If a potential preamble is detected, the system transitions to *Check* mode. If no preamble is found within a predefined timeout period, it transitions to *Rotate* mode.
- **Check mode:** The algorithm verifies that the subsequent data matches the frame criteria to prevent false positives before fully committing to the lock.
- **Lock mode:** The search window jumps by a fixed space equal to the frame length to check for the next preamble. If it is not found, a `max_misses` counter is incremented. If this counter overflows, the system assumes a Costas loop phase slip has occurred and transitions to *Rotate* mode.
- **Rotate mode:** Triggered by the `max_misses` overflow or search timeout, this state sends an asynchronous message to the upstream rotator block to shift the constellation phase by 90° . Afterward, it immediately returns to *Search* mode.

Performance Limitations and Latency

While this single-branch approach successfully reduces CPU load, it introduces a critical disadvantage regarding the Bit Error Rate (BER) and Frame Error Rate (FER). Because the message to rotate the constellation is asynchronous, the feedback loop suffers from pipeline latency.

When a phase slip occurs during *Lock* mode, the system expects a frame but fails to find the preamble. The `max_misses` variable increments, and upon overflowing, triggers the *Rotate* state. By the time the asynchronous message reaches the rotation block and the newly rotated symbols propagate through the **Viterbi** decoder, several frames of valid data have already been corrupted.

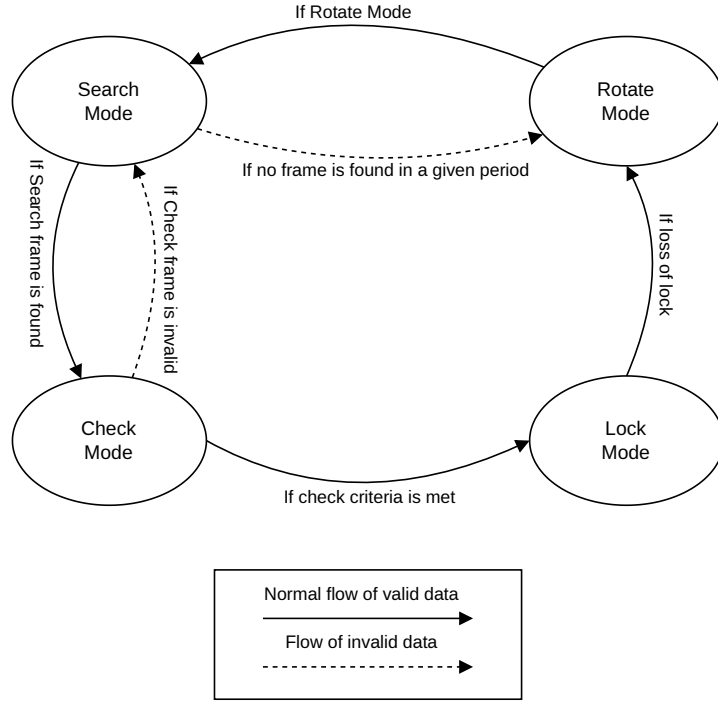


Figure C.2 / State machine diagram for the single-branch asynchronous flywheel algorithm.

Specifically, data is trapped in the Viterbi input buffer, the decoder's history window, the output buffer, and the synchronizer's own input buffer. In practice, this architectural latency results in a systematic loss of 5 consecutive frames every time the Costas loop slips.

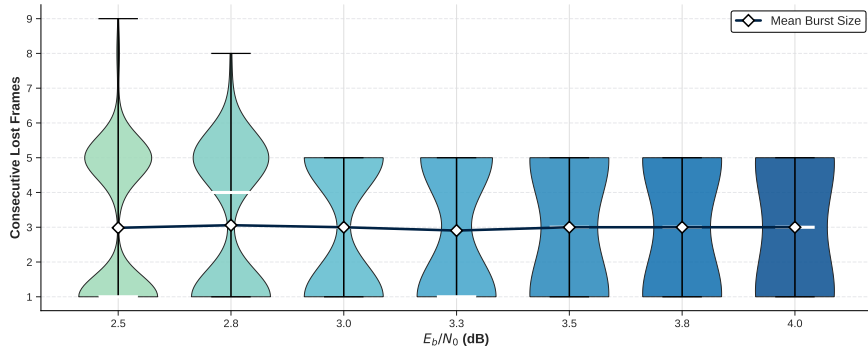


Figure C.3 / Distribution of Sync Loss Burst Sizes using the asynchronous feedback architecture (Doppler Soft 5m).

To mitigate this burst loss, setting `max_misses = 0` was considered, which would theoretically reduce the loss from 5 to 4 frames by reacting instantly to a single missed ASM. However, empirical testing showed that a single bit error within the 32-bit ASM due to channel noise is statistically more probable than a genuine Costas loop phase slip. A `max_misses` value of 0 causes the system to erroneously rotate the phase due to minor noise, destroying the lock entirely. Consequently, `max_misses` must be maintained at 1 or higher.

Ultimately, while the single-branch asynchronous algorithm is computationally lightweight, the inherent pipeline latency and resulting burst frame losses make it unsuitable for links operating at low E_b/N_0 , justifying the shift to the dual-branch feed-forward architecture described in the main text.

References

Back-references to the pages where the publication was cited are given by .

- [1] CCSDS. TM Synchronization and Channel Coding. Consultative Committee for Space Data Systems. **2023**.
URL: <https://public.ccsds.org/Pubs/131x0b5.pdf> , , , , 
- [2] CCSDS. TM Space Data Link Protocol. Consultative Committee for Space Data Systems. **2023**.
URL: <https://public.ccsds.org/Pubs/132x0b3.pdf> , 
- [3] Dani Estévez. gr-satellites: A GNU Radio OOT module for CubeSat telemetry decoding. <https://github.com/daniestevez/gr-satellites>. Documentation available at <https://gr-satellites.readthedocs.io/en/latest/>. **2015** , , 
- [4] David Todd Miller. gr-HighDataRate_Modem: GNU Radio module for high data rate modems. https://github.com/DavidToddMiller/gr-HighDataRate_Modem. **2022** , 
- [5] Irfan Ali, Naofal Al-Dhahir, and John E. Hershey. Doppler characterization for LEO satellites. *IEEE Transactions on Communications*, **1998**.
DOI: [10.1109/26.662636](https://doi.org/10.1109/26.662636) , 
- [6] Dani Estévez. Real time Doppler correction with GNU Radio. **2022**.
URL: <https://destevez.net/2022/07/real-time-doppler-correction-with-gnu-radio/> , 
- [7] Wolfgang Nagele. gr-gpredict-doppler: GNU Radio block for Gpredict Doppler shift correction. <https://github.com/wnagele/gr-gpredict-doppler>. **2015** , 
- [8] CCSDS. TM Synchronization and Channel Coding—Summary of Concept and Rationale. Green Book. CCSDS, **2020**.
URL: <https://ccsds.org/Pubs/130x1g3e1.pdf> , , , , , , , , 
- [9] Gérard Maral and Michel Bousquet. Satellite Communications Systems: Systems, Techniques and Technology. 5th. Wiley, **2009** , 
- [10] Jan Šustr. 10GHz LNB TCXO Technical Manual. **2024**.
URL: [Available%20from%20Passion%20Radio.%20URL:%20%5Curl%7Bhttps://www.passion-radio.com/satellite-qo-100/converter-4](https://www.passion-radio.com/satellite-qo-100/converter-4) , 
- [11] CCSDS. Radio Frequency and Modulation Systems—Part 1: Earth Stations and Spacecraft. Consultative Committee for Space Data Systems. **2023**.
URL: <https://ccsds.org/Pubs/401x0b32.pdf> , 
- [12] Bernard Sklar. Digital Communications: Fundamentals and Applications. 2nd. Prentice Hall, **2001** 
- [13] Antonio Artés Rodríguez, Fernando Pérez González, et al. Comunicaciones Digitales. Pearson Educación, **2007** , 
- [14] John G. Proakis and Masoud Salehi. Digital Communications. 5th. McGraw-Hill, **2007** 
- [15] Øystein Lofaldli. gr-ccsds: GNU Radio implementation of the CCSDS space data system standards. <https://github.com/lofaldli/gr-ccsds>. **2016** 
- [16] David A. Vallado. Fundamentals of Astrodynamics and Applications. 2nd. Microcosm Press, **2001** 

- [17] Floyd M. Gardner. Phaselock Techniques. 3rd. John Wiley & Sons, Inc., **2005**
16, 17, 18, 19
- [18] Daniel Estévez. Decoding the QO-100 Multimedia Beacon with GNU Radio. **2022**.
URL: <https://destevez.net/2022/05/decoding-the-qo-100-multimedia-beacon-with-gnu-radio/>
(visited on 06/24/2026) 30
- [19] Daniel Estévez. QO-100 Multimedia Beacon GNU Radio Flowgraph. **2022**.
URL: <https://gist.github.com/daniestevez/d7a969170c9c8a13070304b728e91f94> (visited on
06/24/2026) 30